

NANYANG TECHNOLOGICAL UNIVERSITY

Data Structures & Algorithms in Python

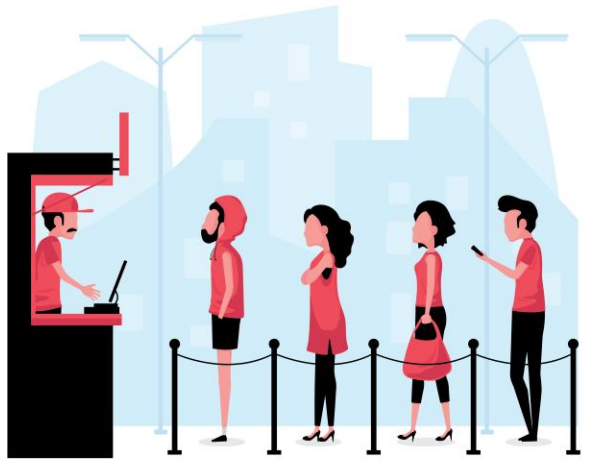
Binary Trees – Lab Answers

College of Computing and Data Science

Stacks & Queues Recap

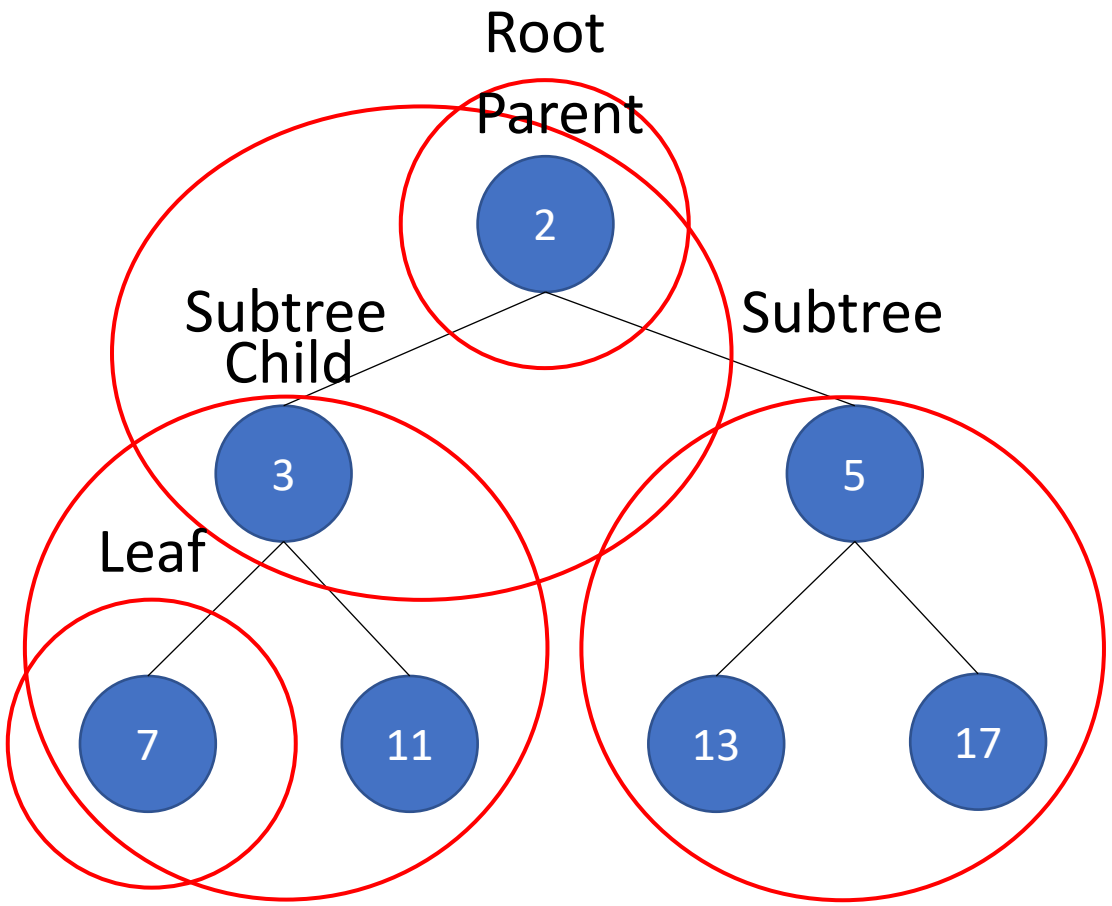
Stacks	Queues
Last-In, First-Out (LIFO)	First-In, First-Out (FIFO)
push() and pop()	enqueue() and dequeue()





Content Copyright Nanyang Technological University

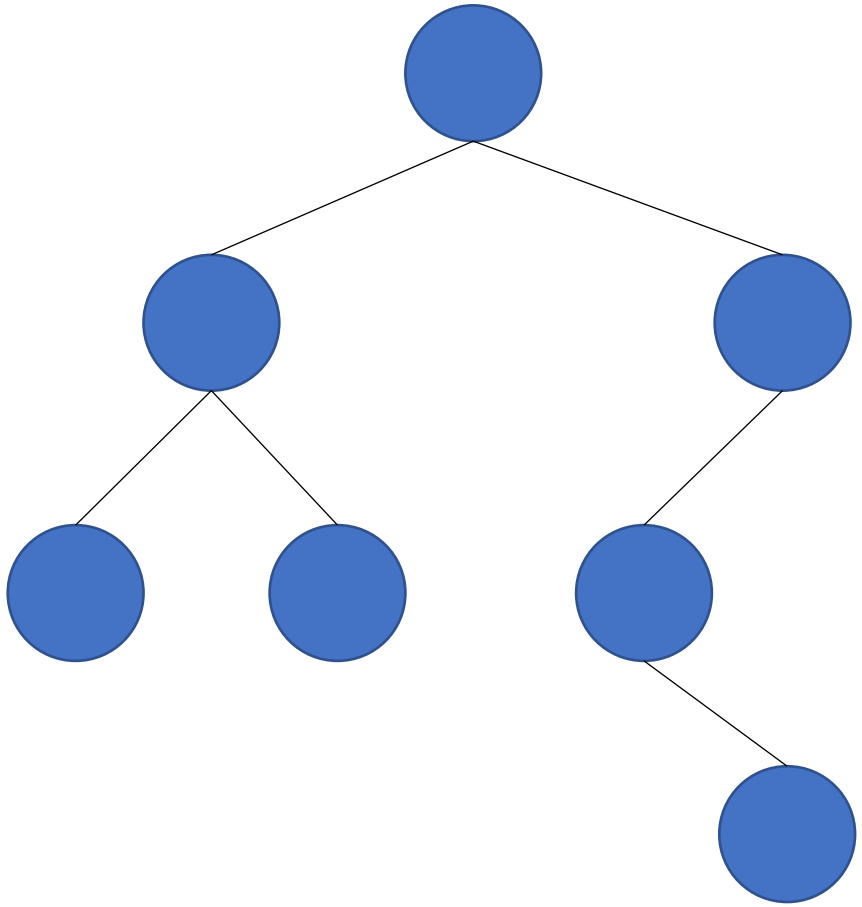
Binary Trees





Content Copyright Nanyang Technological University

Binary Trees

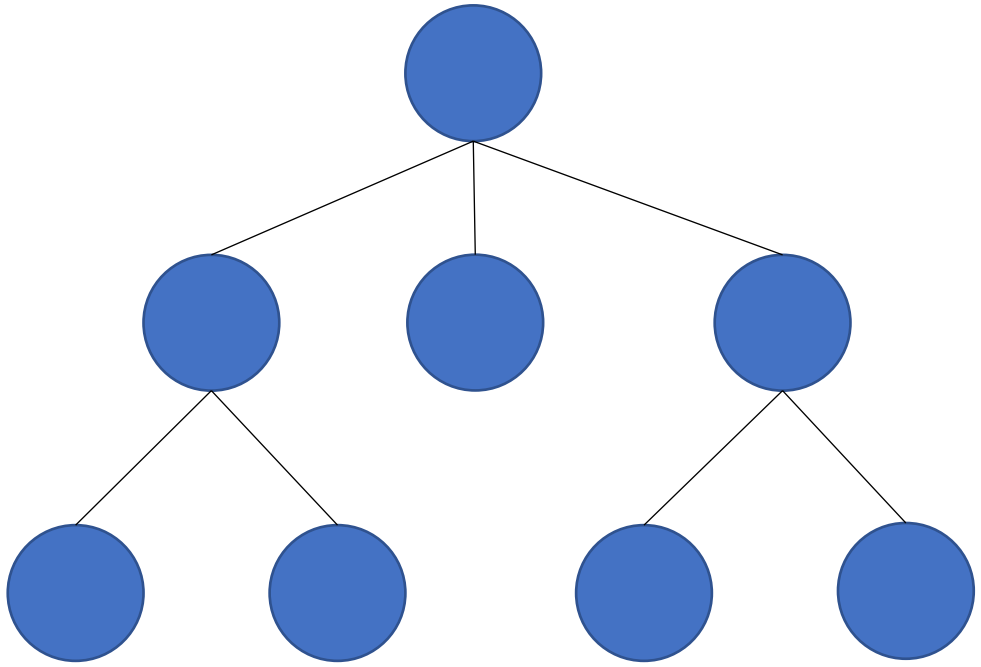


?



Content Copyright Nanyang Technological University

Binary Trees

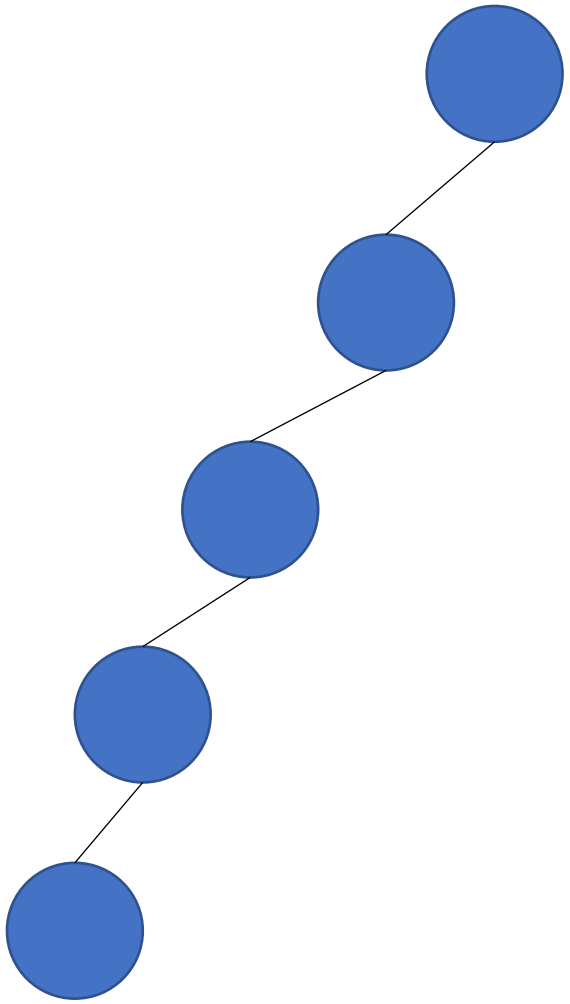


?



Content Copyright Nanyang Technological University

Binary Trees

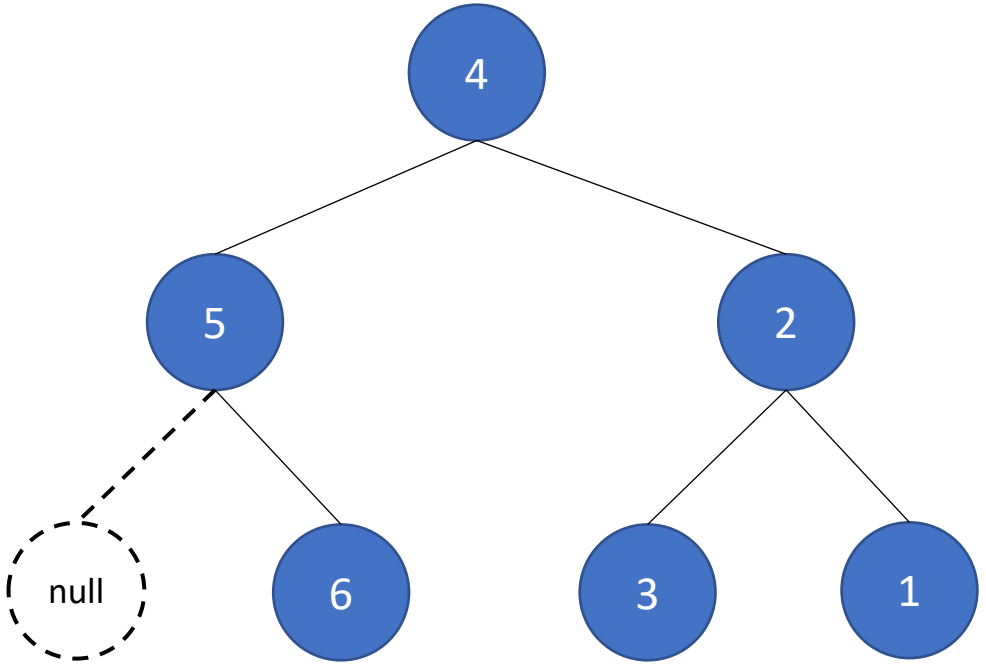


?



Content Copyright Nanyang Technological University

Binary Trees

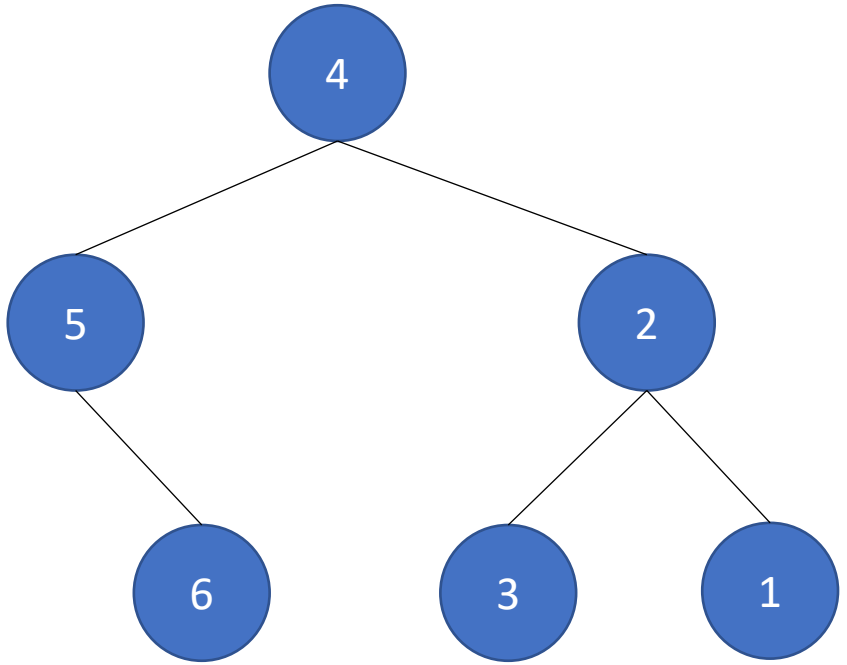


[4, 5, 2, null, 6, 3, 1]

```
class TreeNode:
    def __init__(self, data=0):
        self.data = data
        self.left = None
        self.right = None
```

Content Copyright Nanyang Technological University

Binary Trees



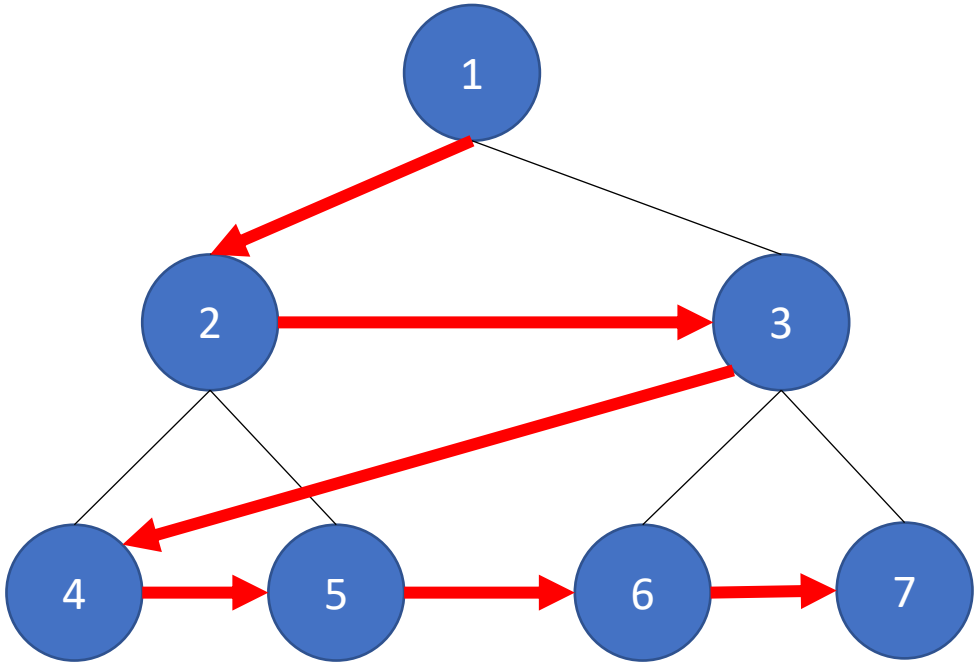

```
class BTNode:
    def __init__(self, item, left=None, right=None):
        self.item = item
        self.left = left
        self.right = right
```

```
root = BTNode(4)
root.left = BTNode(5)
root.right = BTNode(2)
root.left.left = None
root.left.right = BTNode(6)
root.right.left = BTNode(3)
root.right.right = BTNode(1)
```

Content Copyright Nanyang Technological University

Tree Traversal

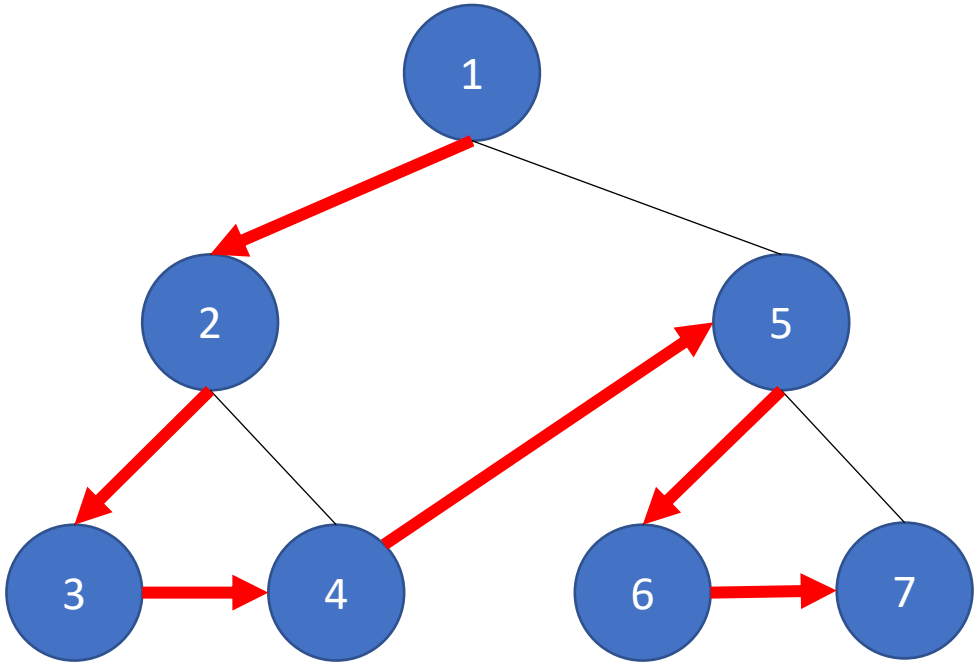
- Breadth-First Search
  - Layer by layer
  - Left to right



Content Copyright Nanyang Technological University

Tree Traversal

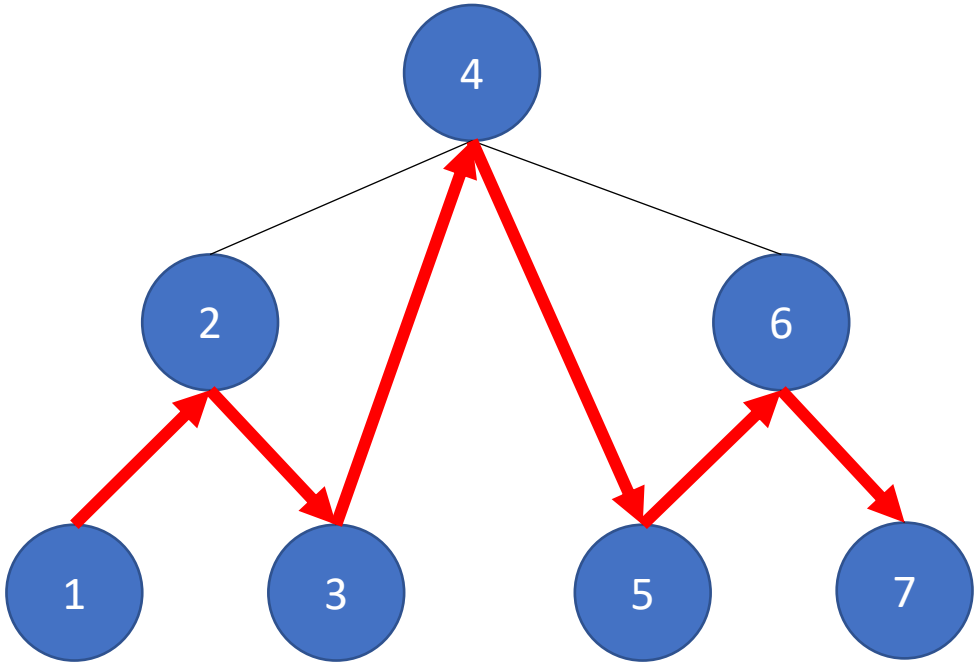
- Depth-First Search
  - Preorder traversal
  - Root, left, right



Content Copyright Nanyang Technological University

Tree Traversal

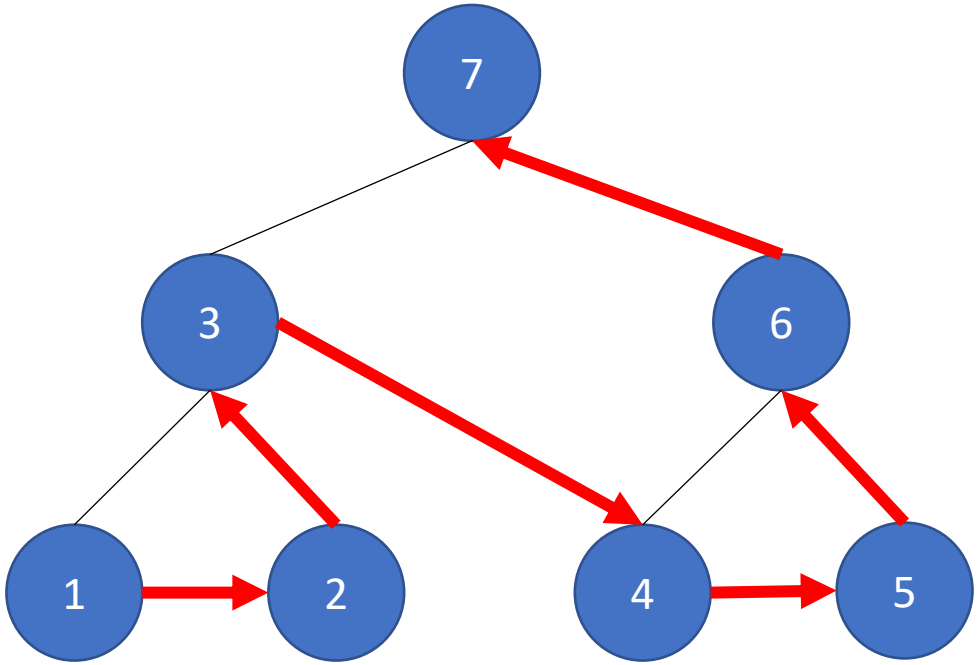
- Depth-First Search
  - In-order traversal
  - Left, root, right



Content Copyright Nanyang Technological University

Tree Traversal

- Depth-First Search
  - Postorder traversal
  - Left, right, root



Content Copyright Nanyang Technological University

Q1 (No AI)

Given the root of a binary tree, mirror the tree by swapping the left and right children of every node.

Example 1:

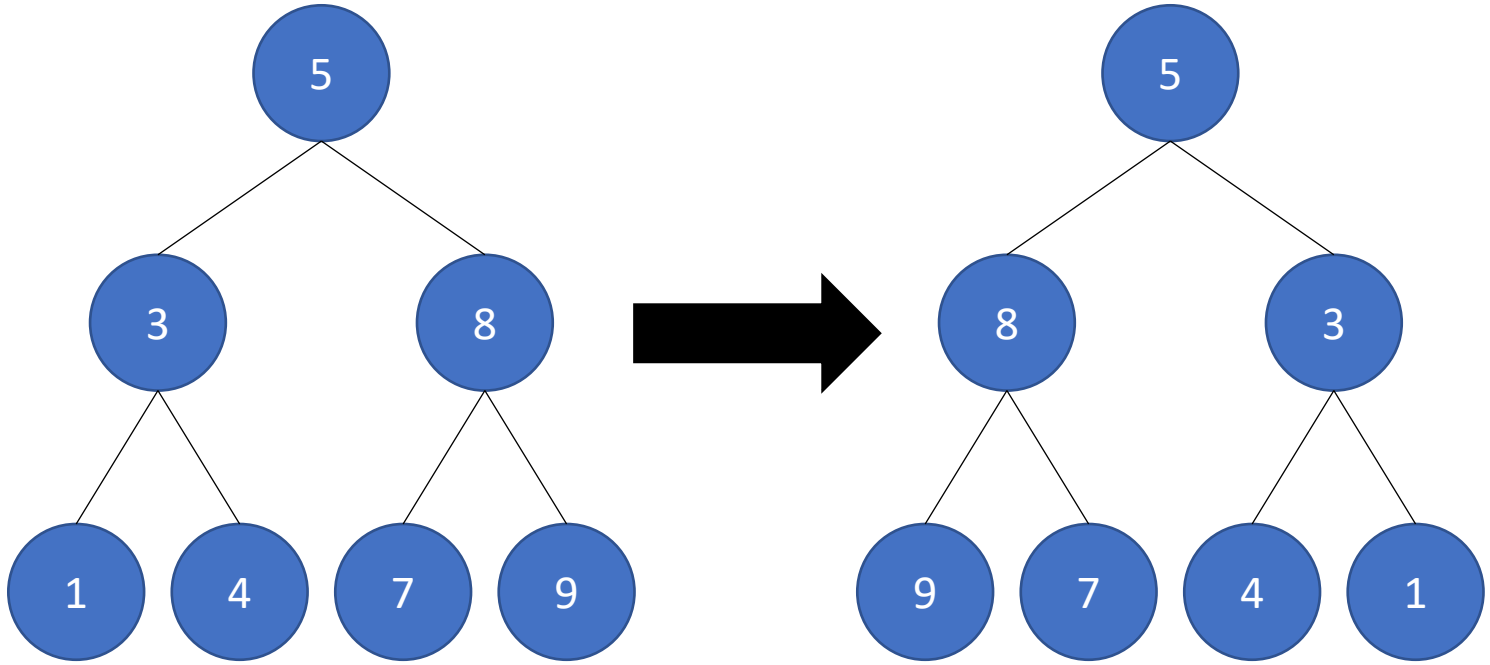
Input: [5, 3, 8, 1, 4, 7, 9]
Output: [5, 8, 3, 9, 7, 4, 1]
Explanation: Every node's left and right children are swapped.

Example 2:

Input: [5, 3, 8]
Output: [5, 8, 3]

Example 3:

Input: [1]
Output: [1]



Content Copyright Nanyang Technological University

Q2 (Normal)

Given the root of a binary tree, identify all nodes that have at least one grandchild (a child of their child).

Return a list of those node values. The order does not matter.

Example 1:

Input: [5, 3, 8, 1, 4]
Output: [5]
Explanation: 5's children are 3 and 8; 3 has children 1 and 4, so 5 has grandchildren.
3's children are leaves, so 3 does not.

Example 2:

Input: [10, 5, 15, 3, 7, 13, 18]
Output: [10]
Explanation: only 10 has grandchildren (3, 7, 13 and 18).
5 and 15 have children but no grandchildren.

Example 3:

Input: [1]
Output: []

Content Copyright Nanyang Technological University

Q3 (Driver)

Write an iterative Python function postOrderIterativeS1() that prints the post-order traversal of a binary search tree using only one temporary stack inside the postOrderIterativeS1()function.

Note that you should only use push() or pop() operations when you add or remove integers from the stack.

Content Copyright Nanyang Technological University

Q4 (Navigator)

Write a recursive Python function identical() to determine whether two binary trees are structurally identical, assuming the two binary trees as tree1 and tree2. This function returns TRUE if two binary trees are structurally identical; otherwise, it returns FALSE.

Note that two binary trees are structurally identical if they are both empty or if they are both non-empty and the left and the right subtrees are similar (they are made of nodes with the same values and arranged in the same way).

Content Copyright Nanyang Technological University

Lab Access Code

SR3-L3-52LK

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer

Given the root of a binary tree, mirror the tree by swapping the left and right children of every node.

Example 1:

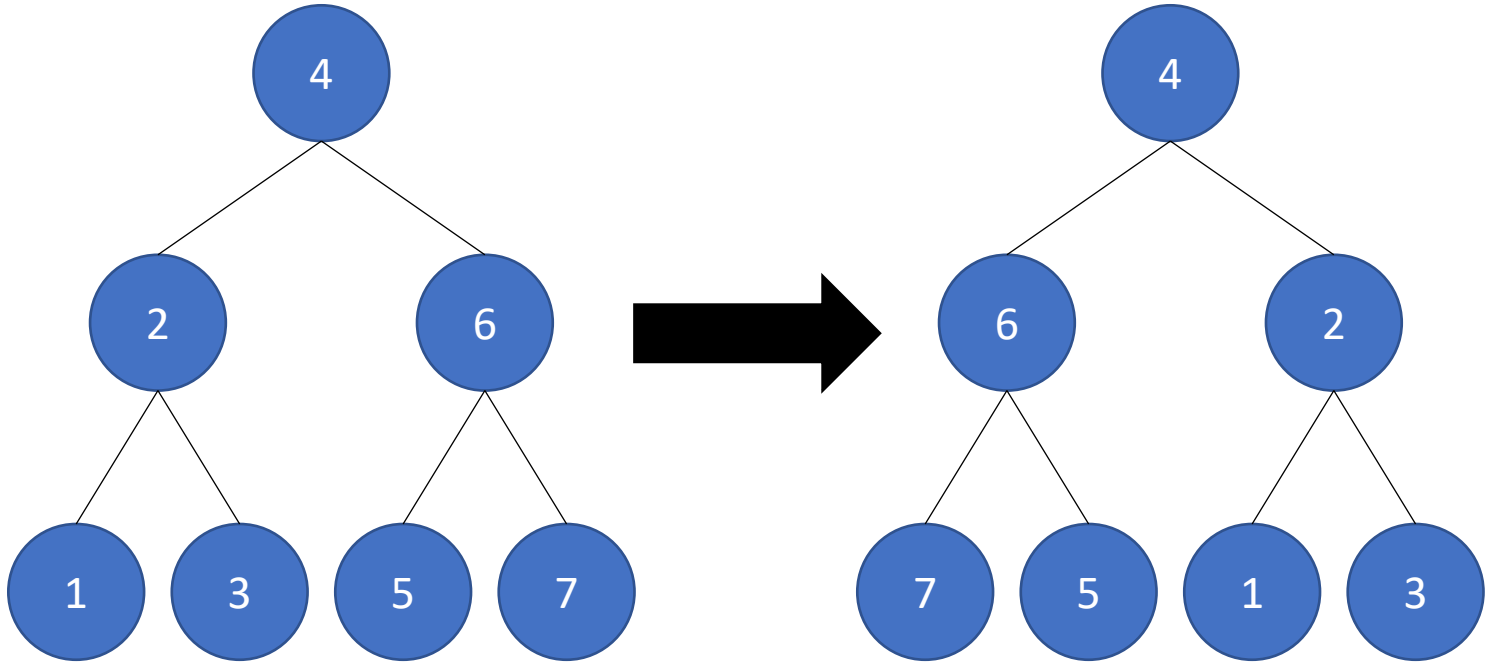
Input: [5, 3, 8, 1, 4, 7, 9]
Output: [5, 8, 3, 9, 7, 4, 1]
Explanation: Every node's left and right children are swapped.

Example 2:

Input: [5, 3, 8]
Output: [5, 8, 3]

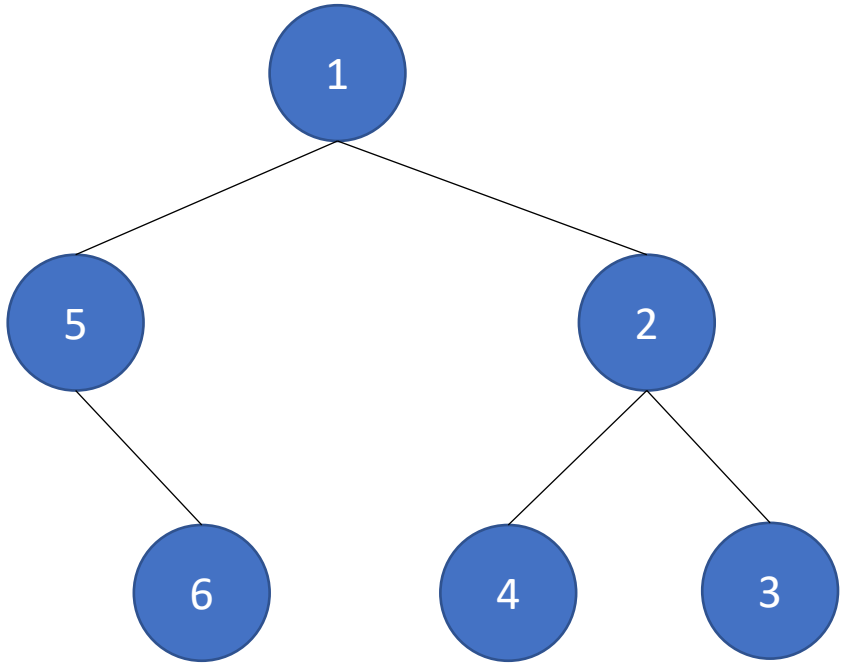
Example 3:

Input: [1]
Output: [1]



Content Copyright Nanyang Technological University

Q1 (No AI) – Answer




```
def mirrorTree(self, root):
    if root is None:
        return None

    root.left, root.right = root.right, root.left

    self.mirrorTree(root.left)
    self.mirrorTree(root.right)

    return root
```

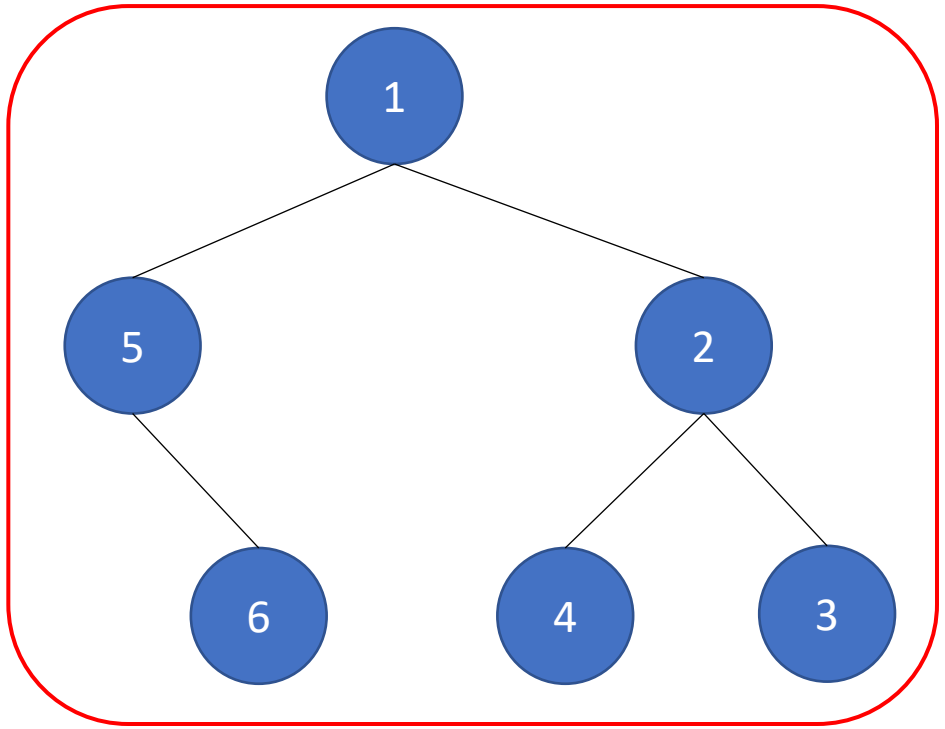
Preorder traversal

```
def mirrorTree(node):
    if node is None:
        return
    mirrorTree(node.left)
    temp = node.left
    node.left = node.right
    node.right = temp
    mirrorTree(node.left)
```

In-order traversal

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer




```
def mirrorTree(self, root):
    if root is None:
        return None

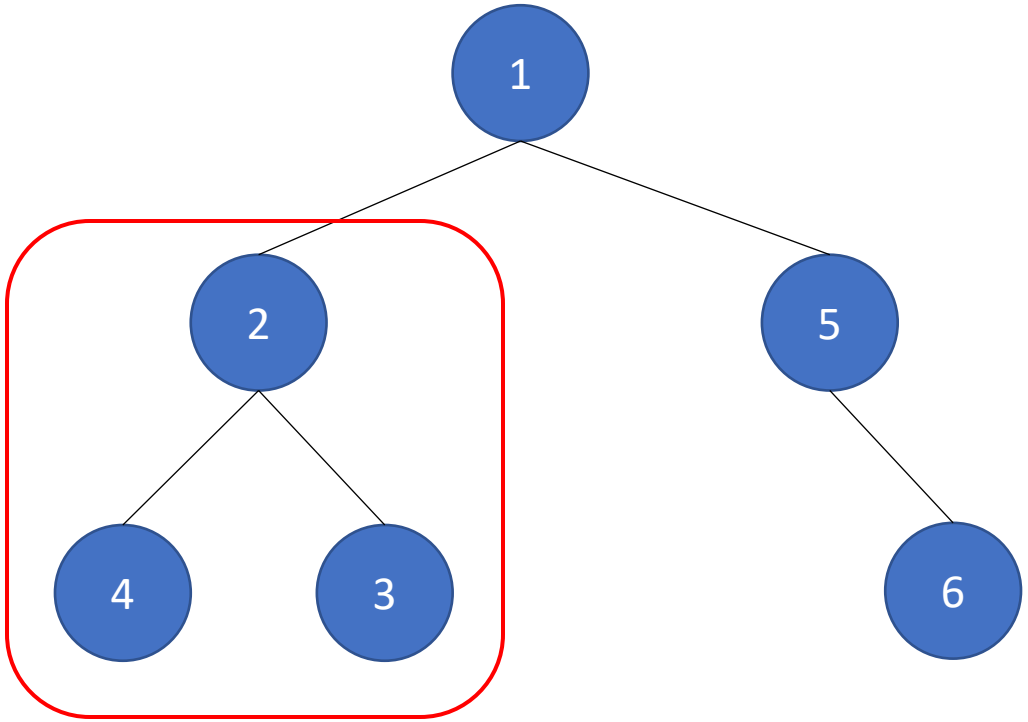
    root.left, root.right = root.right, root.left

    self.mirrorTree(root.left)
    self.mirrorTree(root.right)

    return root
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer




```
def mirrorTree(self, root):
    if root is None:
        return None

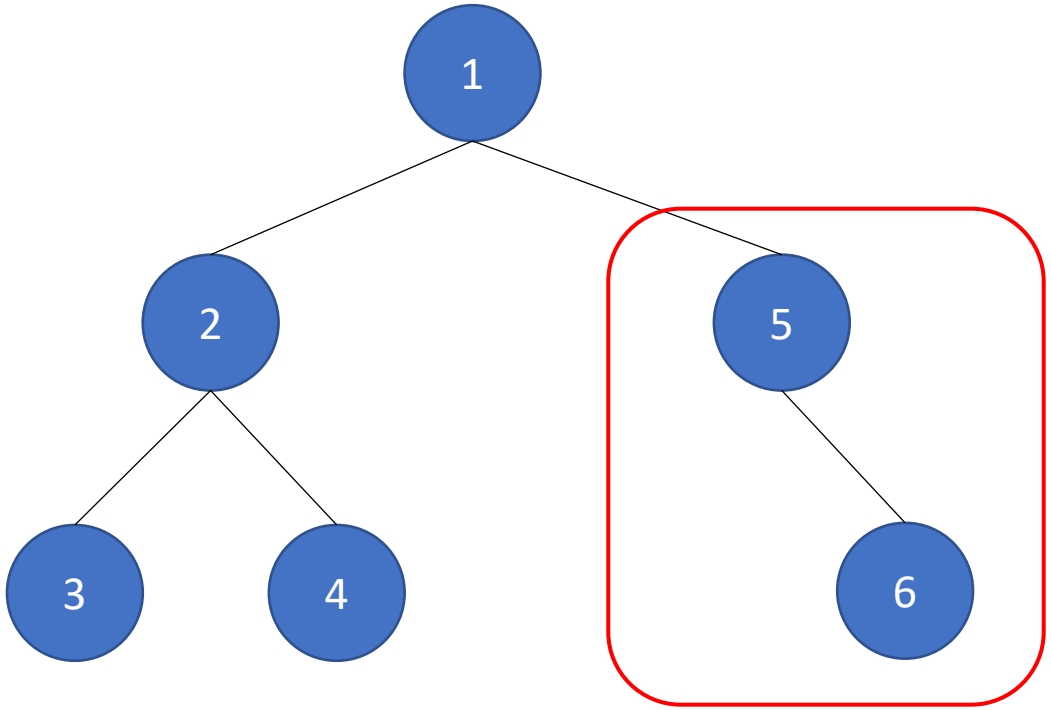
    root.left, root.right = root.right, root.left

    self.mirrorTree(root.left)
    self.mirrorTree(root.right)

    return root
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer




```
def mirrorTree(self, root):
    if root is None:
        return None

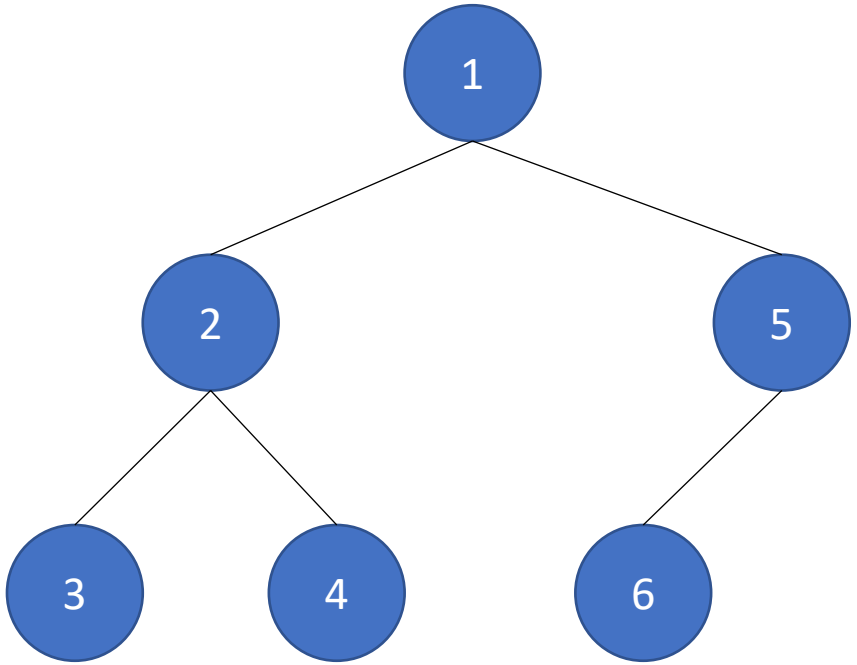
    root.left, root.right = root.right, root.left

    self.mirrorTree(root.left)
    self.mirrorTree(root.right)

    return root
```

Content Copyright Nanyang Technological University

Q1 (No AI) – Answer




```
def mirrorTree(self, root):
    if root is None:
        return None

    root.left, root.right = root.right, root.left

    self.mirrorTree(root.left)
    self.mirrorTree(root.right)

    return root
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

Given the root of a binary tree, identify all nodes that have at least one grandchild (a child of their child).

Return a list of those node values. The order does not matter.

Example 1:

Input: [5, 3, 8, 1, 4]
Output: [5]
Explanation: 5's children are 3 and 8; 3 has children 1 and 4, so 5 has grandchildren.
3's children are leaves, so 3 does not.

Example 2:

Input: [10, 5, 15, 3, 7, 13, 18]
Output: [10]
Explanation: only 10 has grandchildren (3, 7, 13 and 18).
5 and 15 have children but no grandchildren.

Example 3:

Input: [1]
Output: []

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

```
def has_grandchild(self, root):
    if root is None:
        return []

    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer

Pop stack and assign as current

```
def has_grandchild(self, root):
    if root is None:
        return []

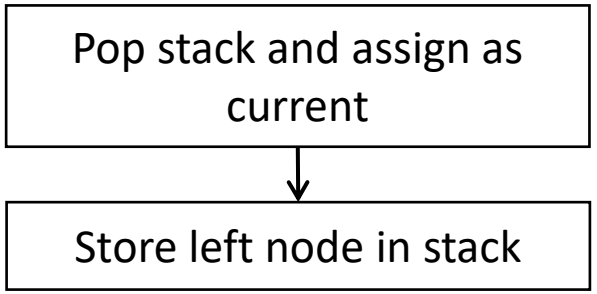
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

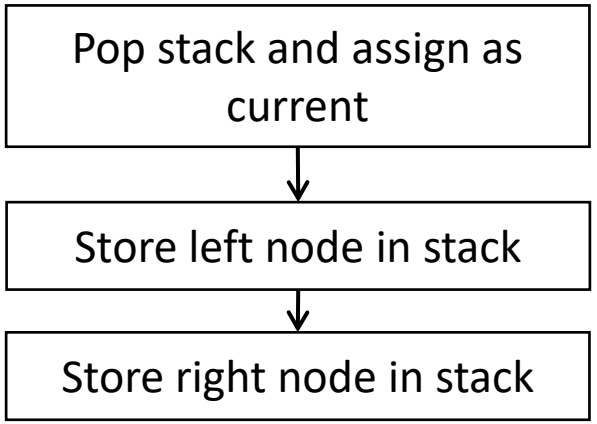
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

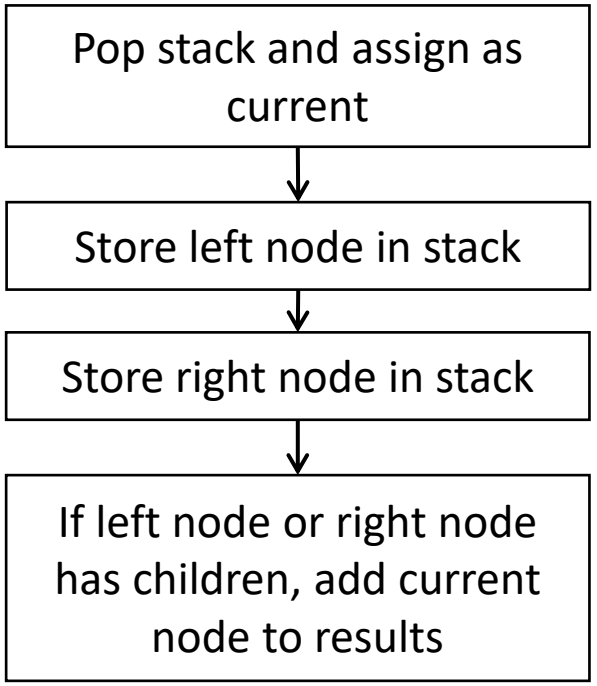
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

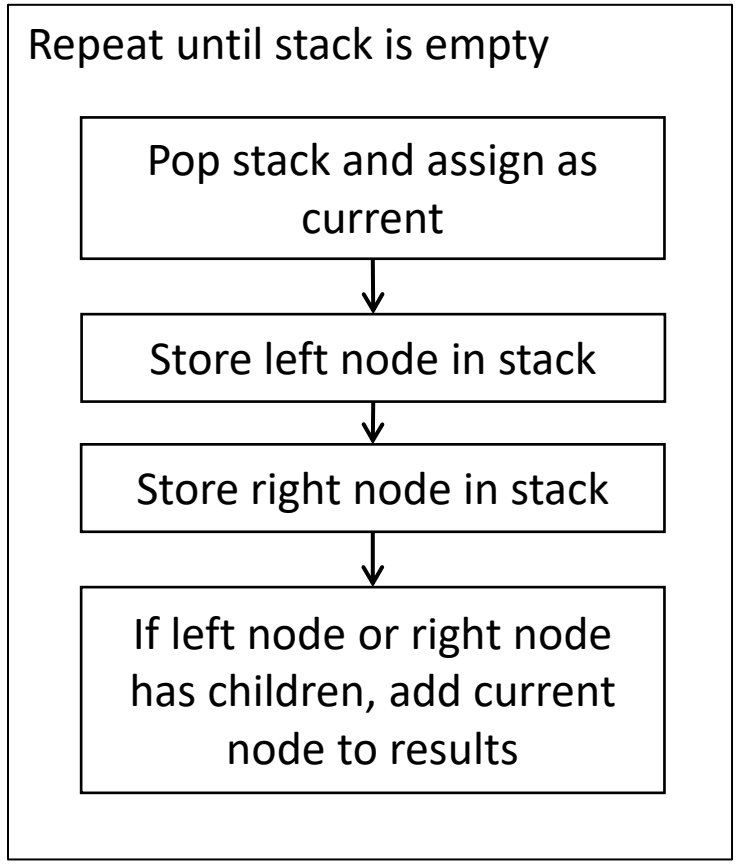
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

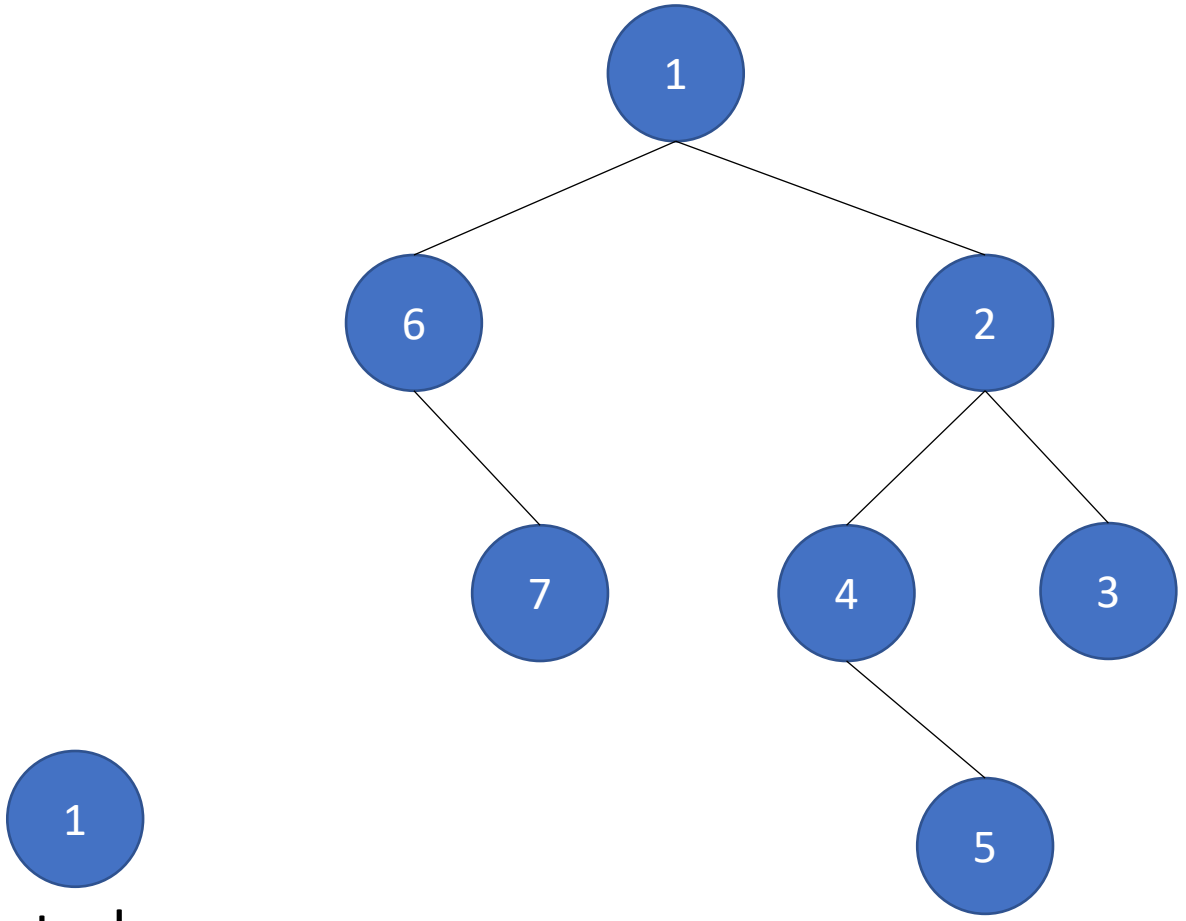
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

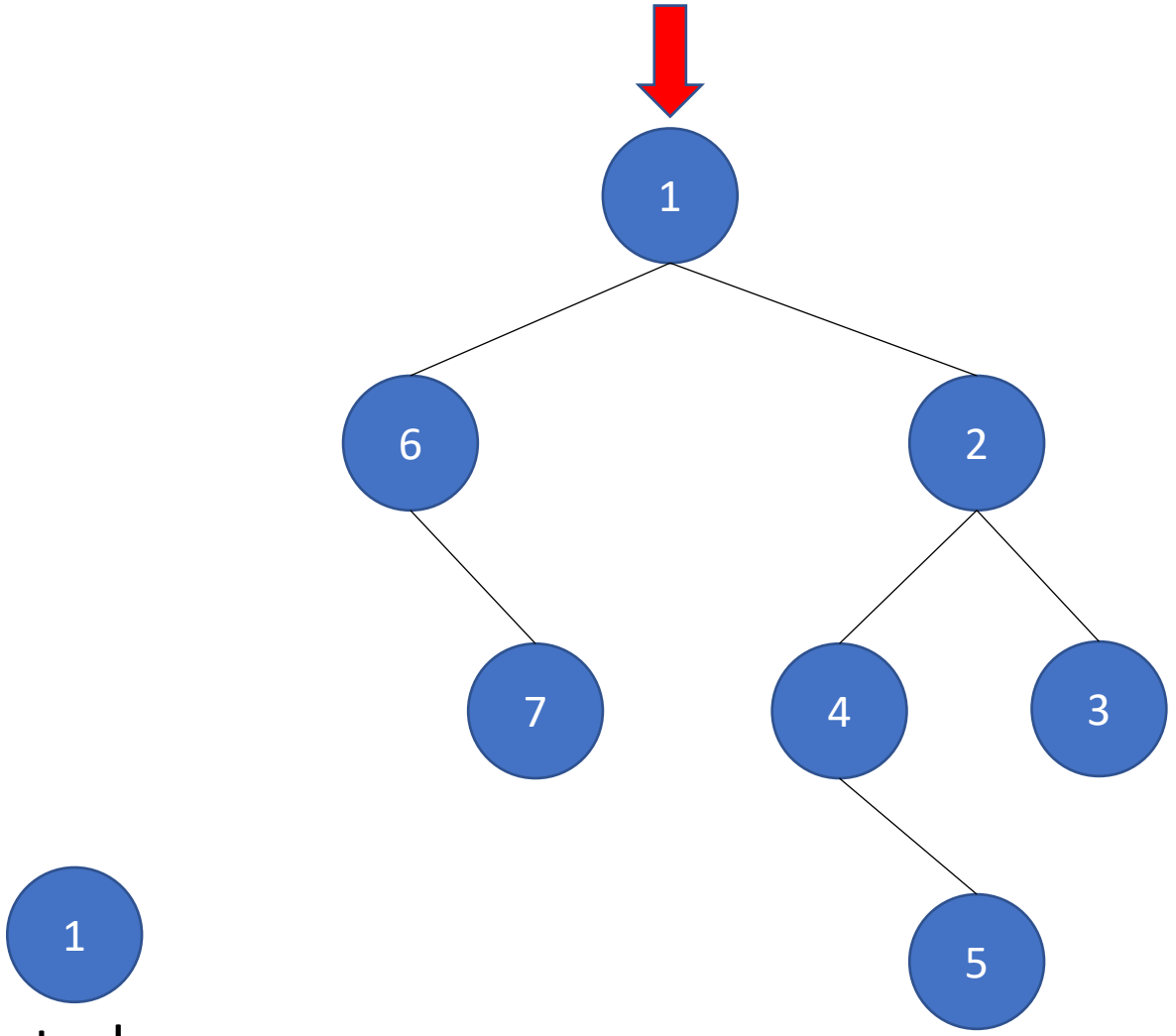
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer





stack

```
def has_grandchild(self, root):
    if root is None:
        return []

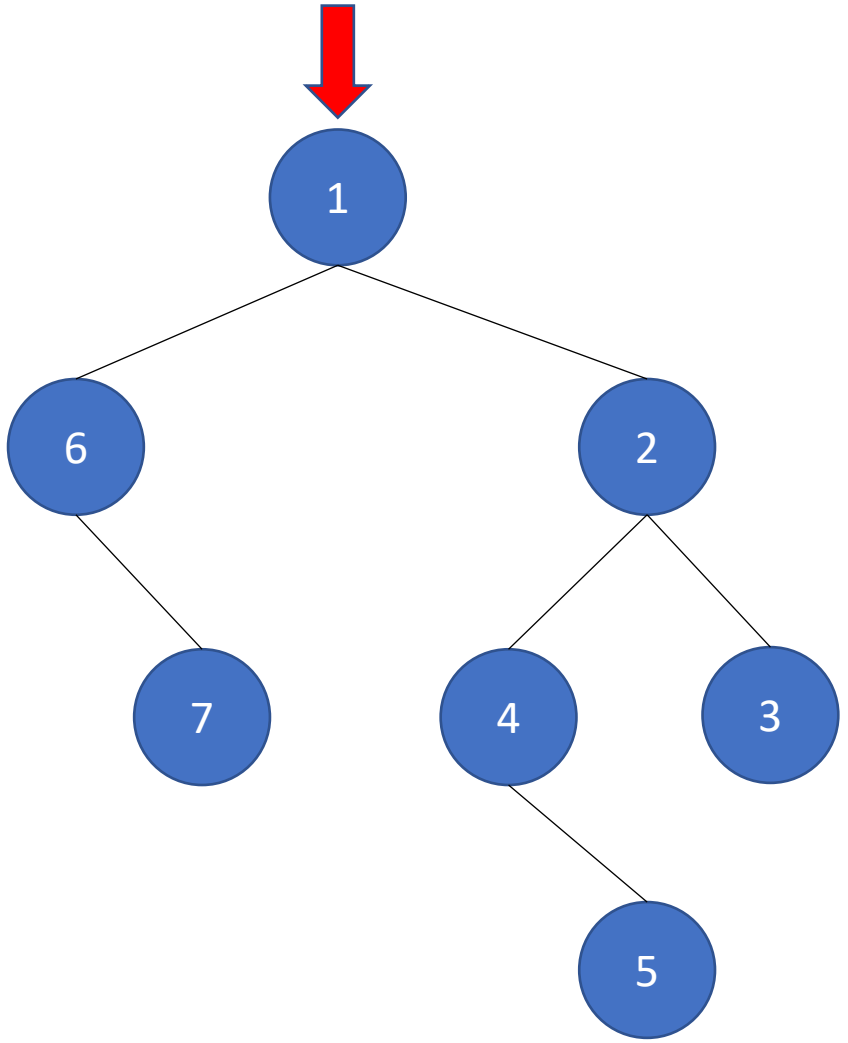
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer



stack

```
def has_grandchild(self, root):
    if root is None:
        return []

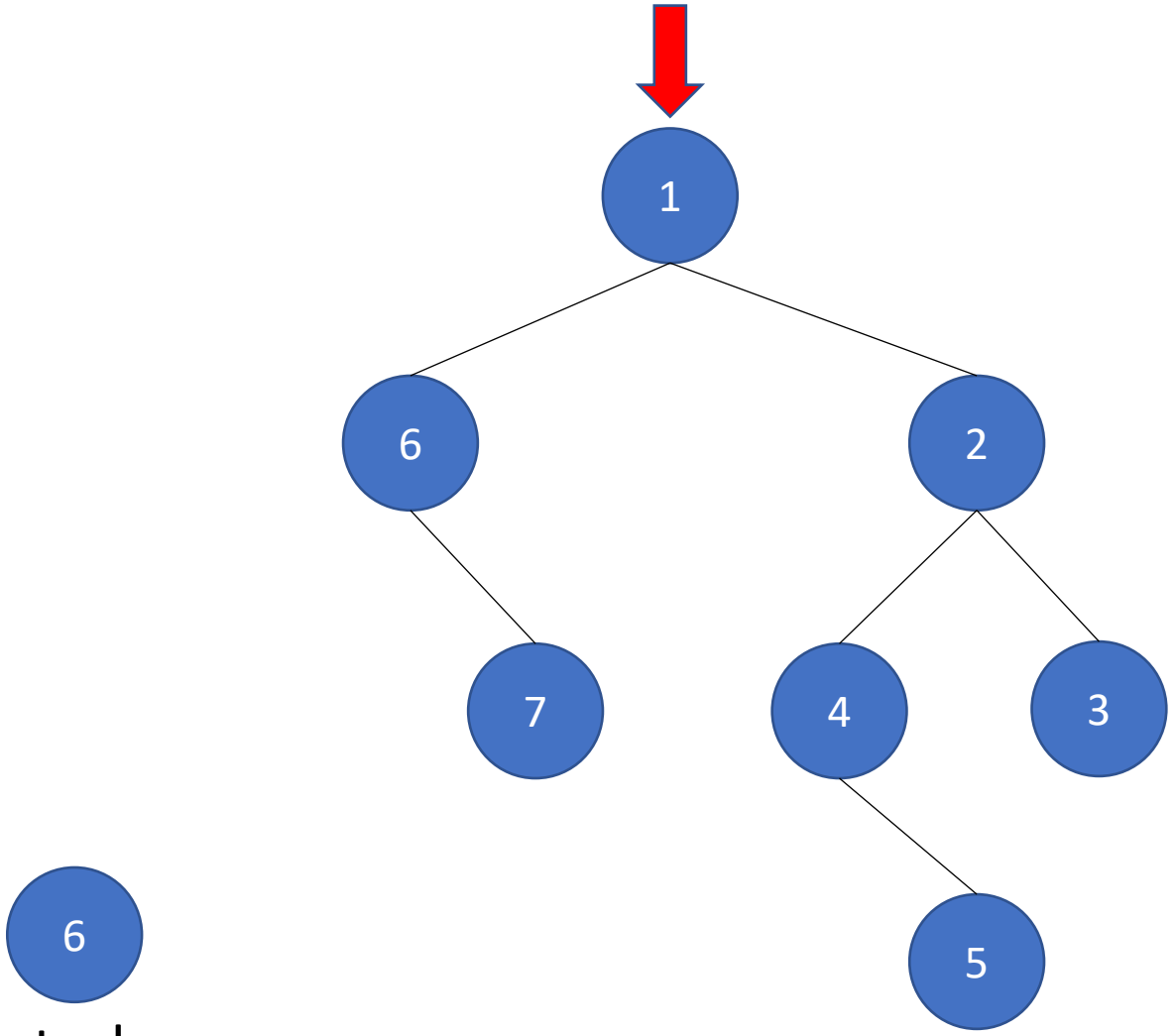
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

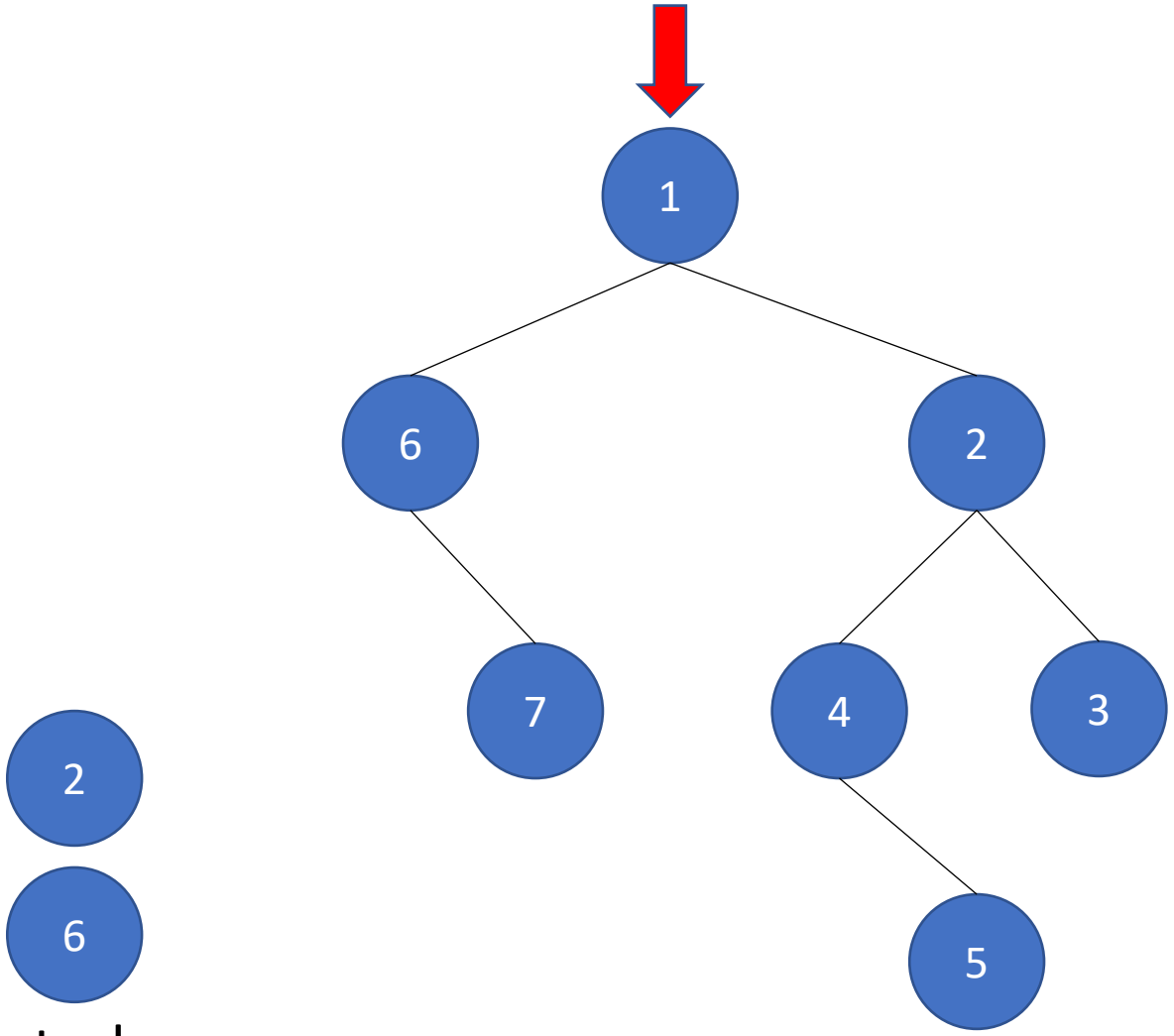
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

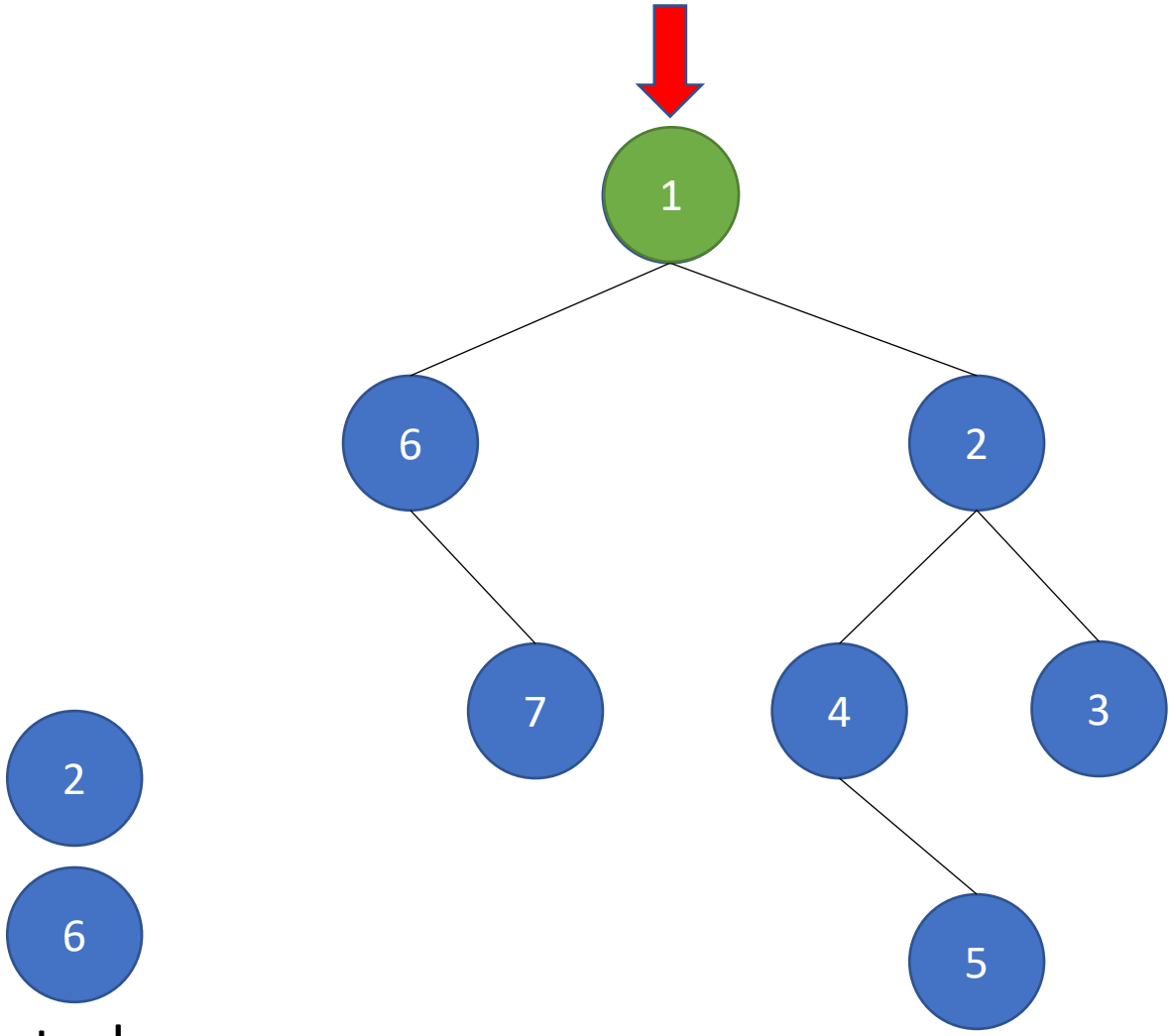
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

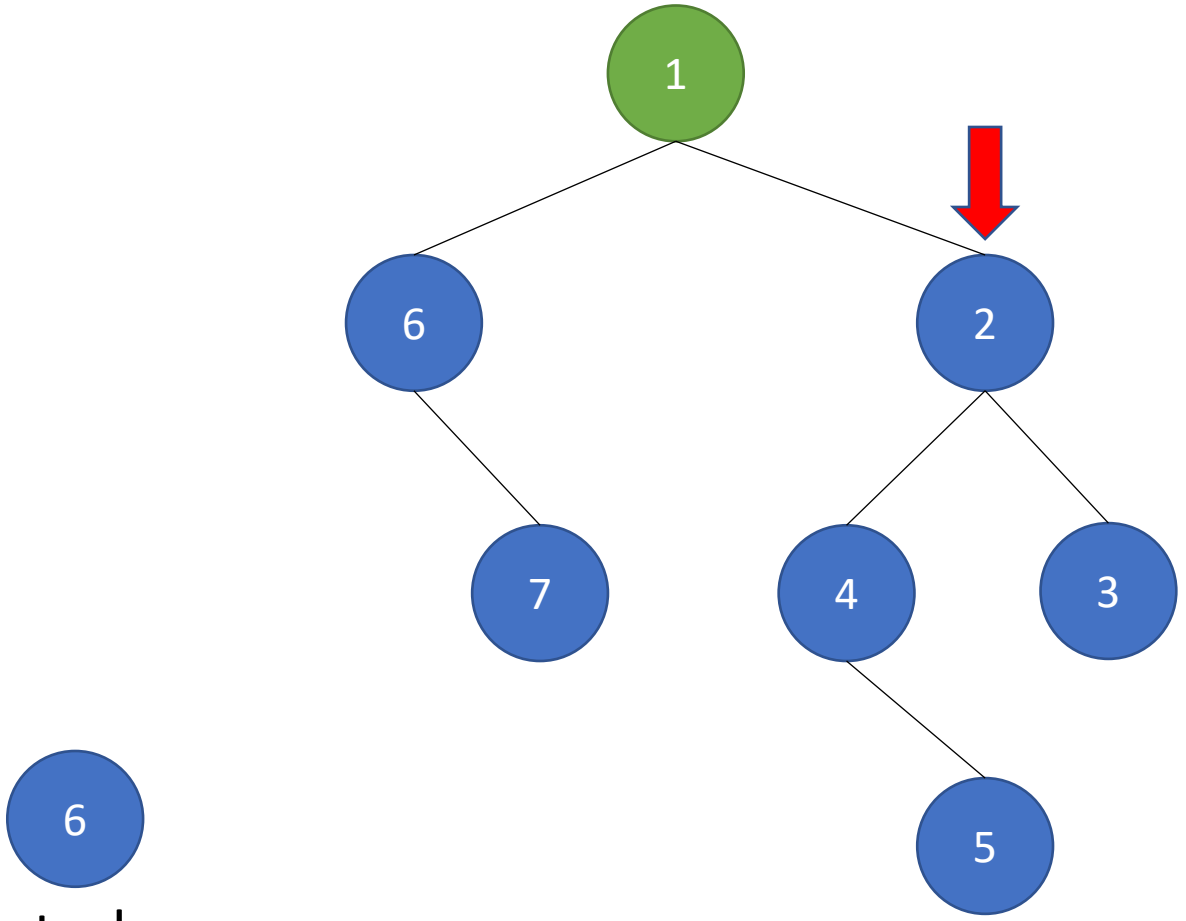
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

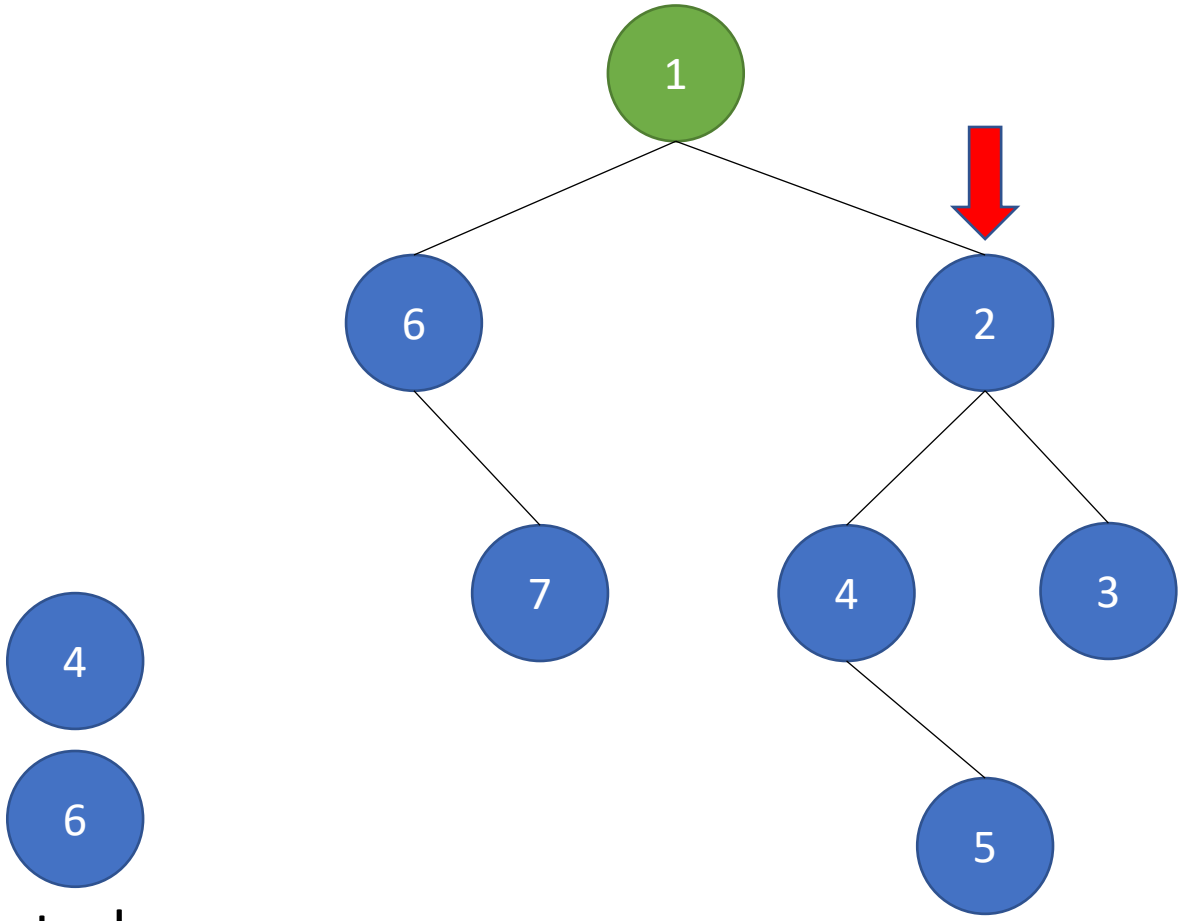
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

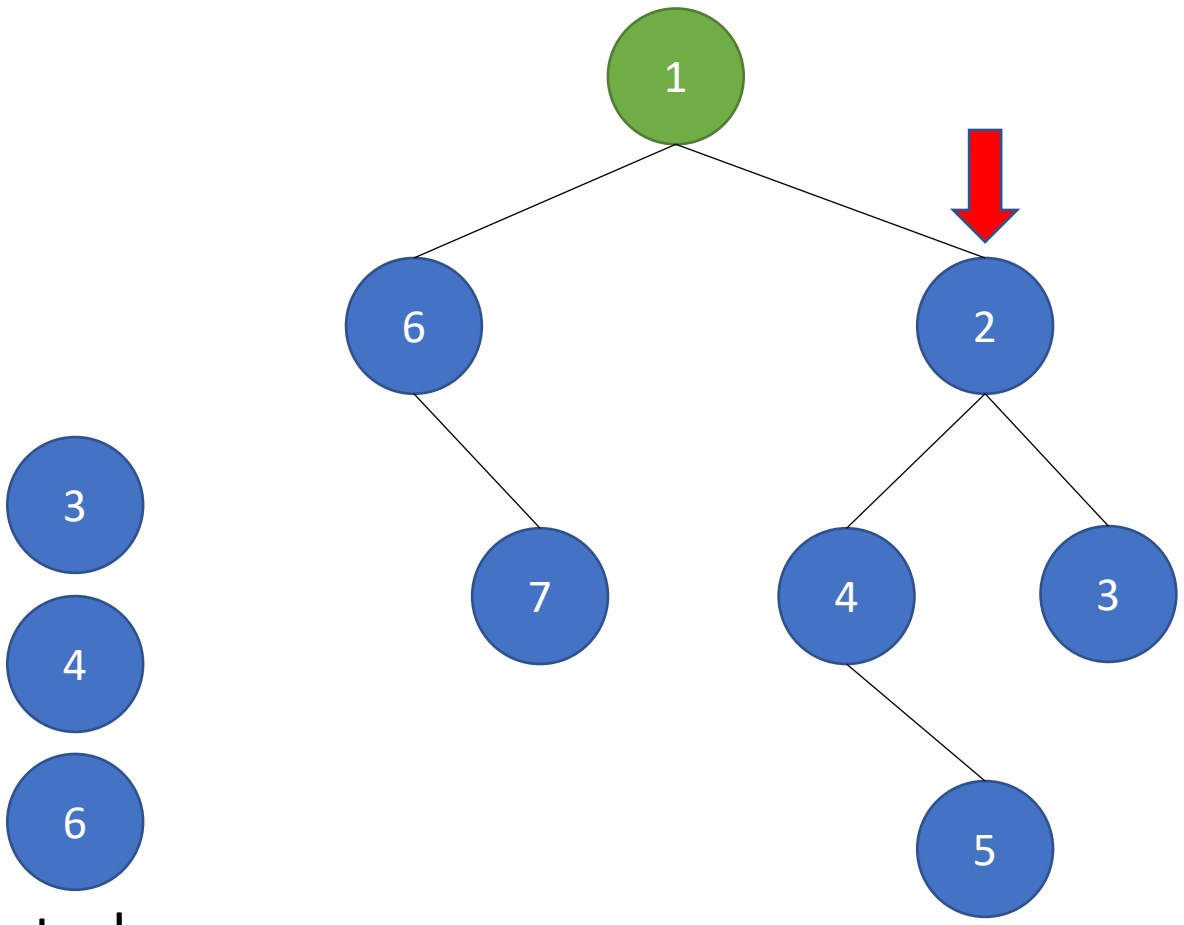
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

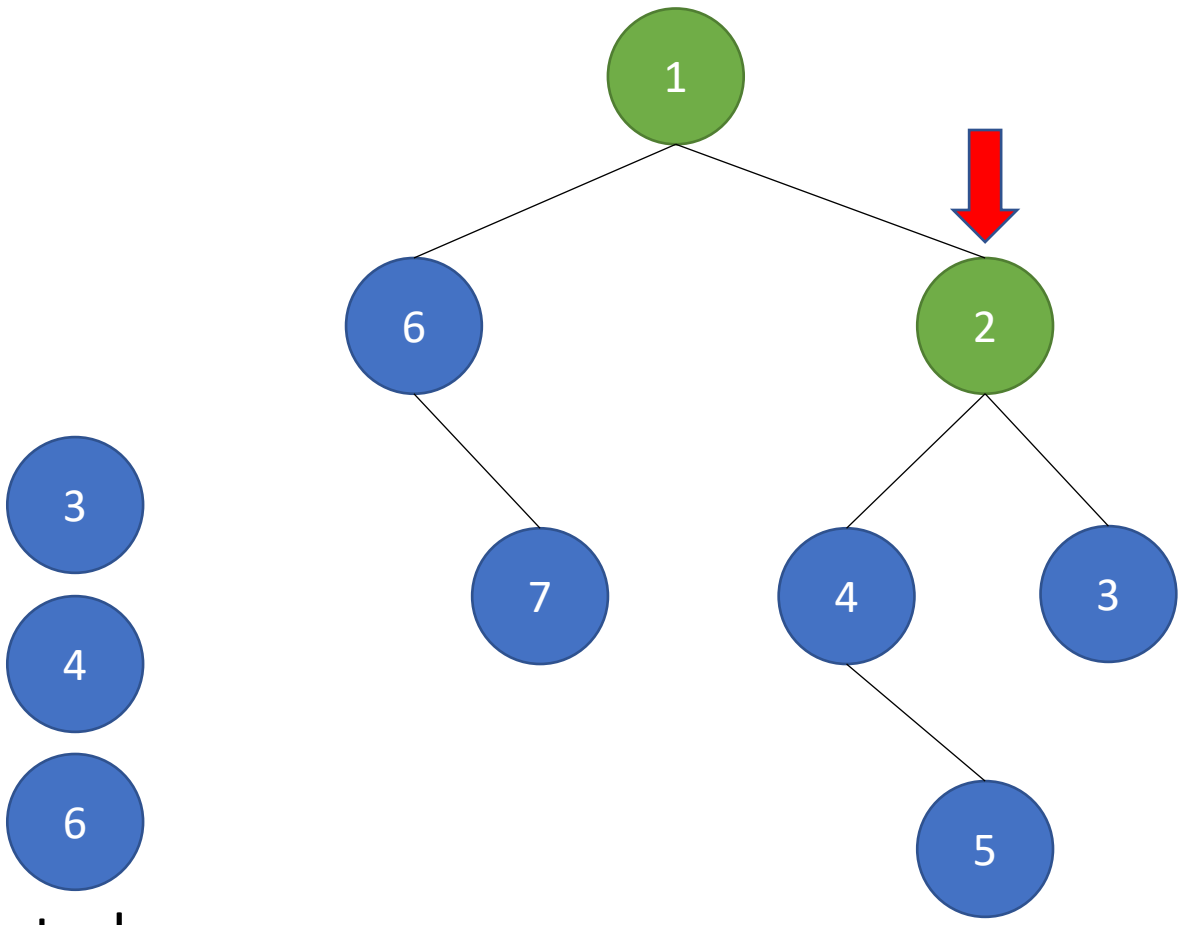
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```python
def has_grandchild(self, root):
    if root is None:
        return []

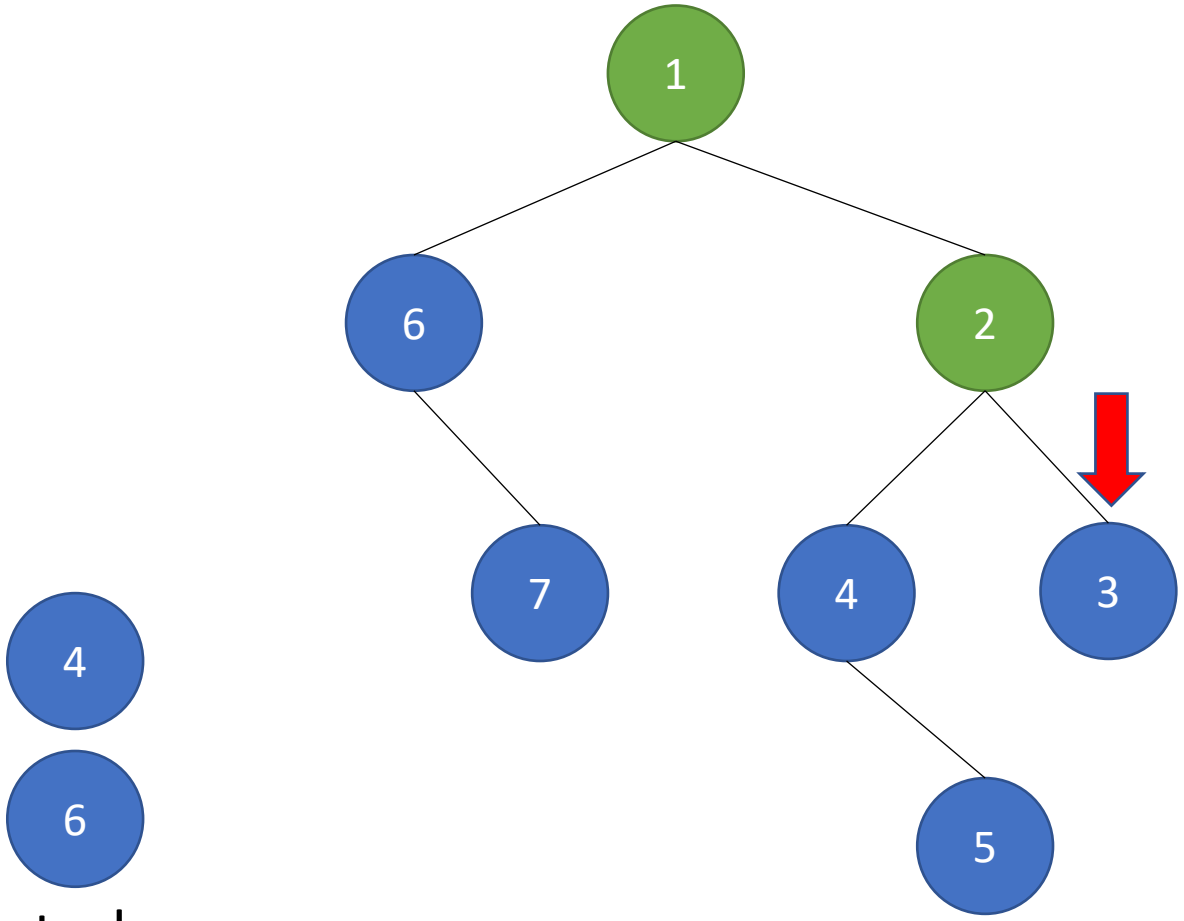
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

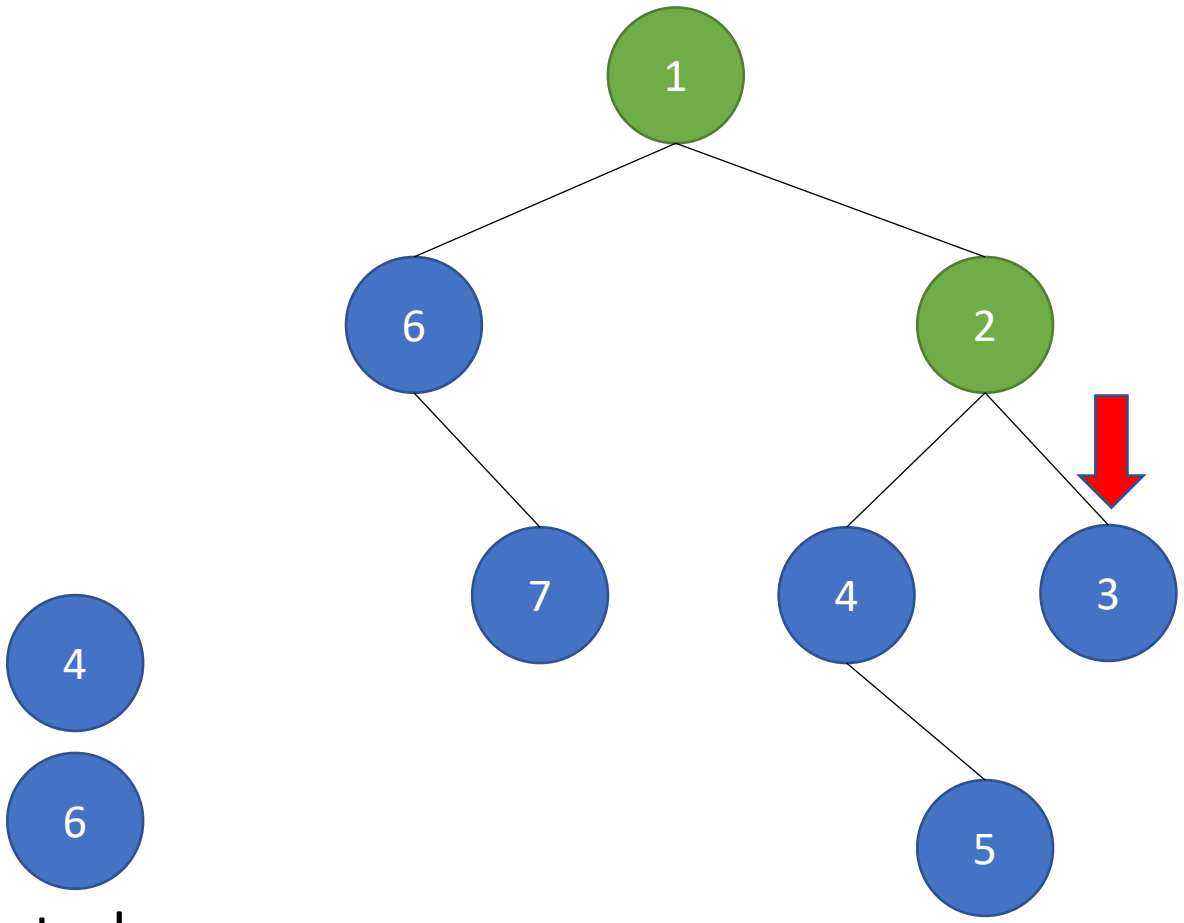
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

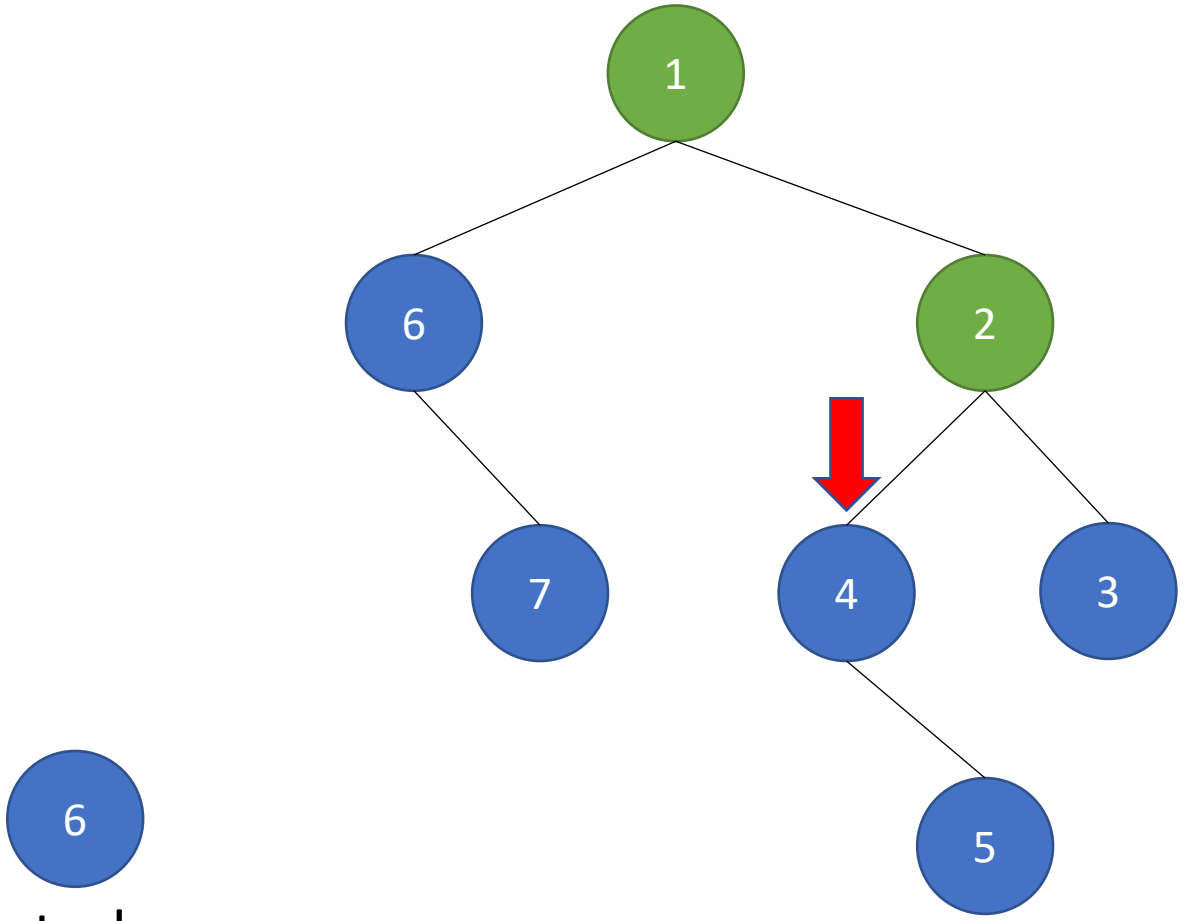
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

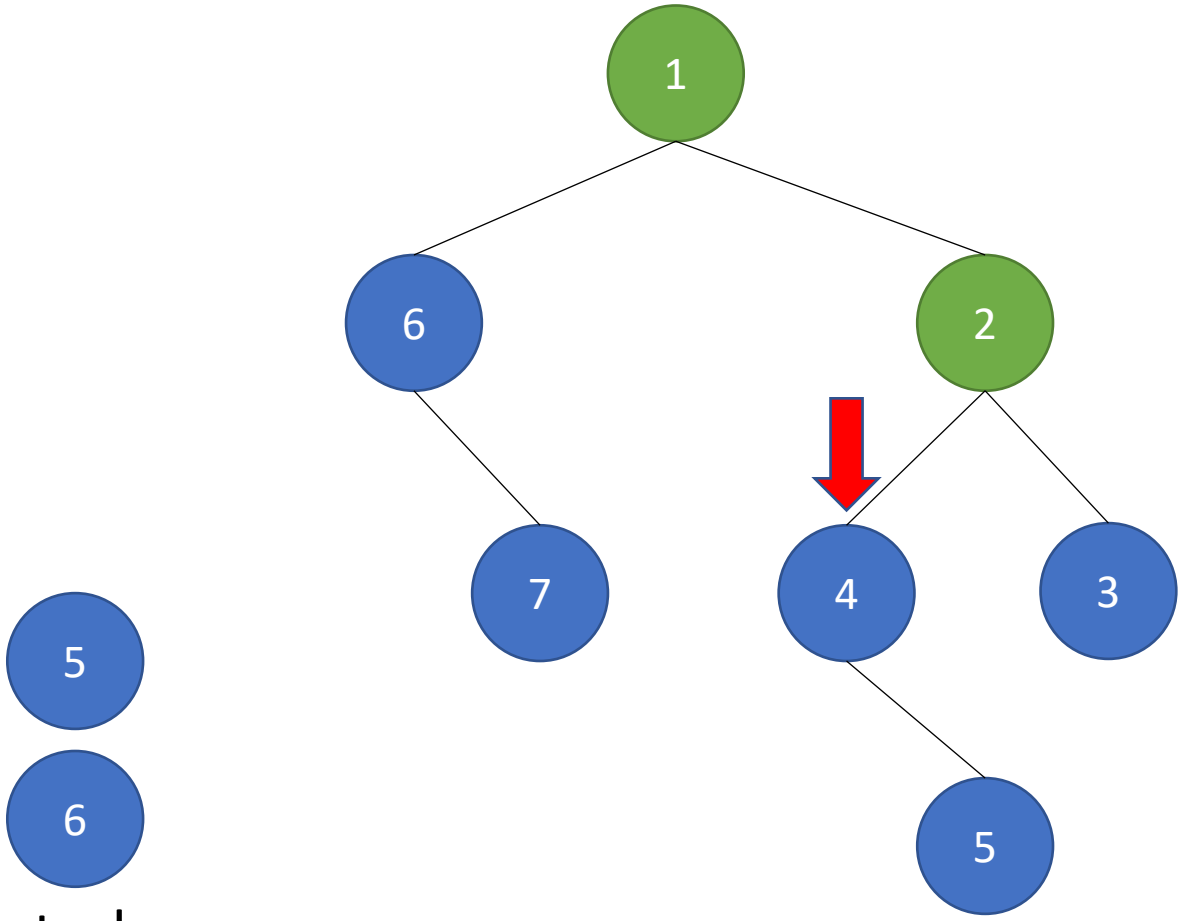
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

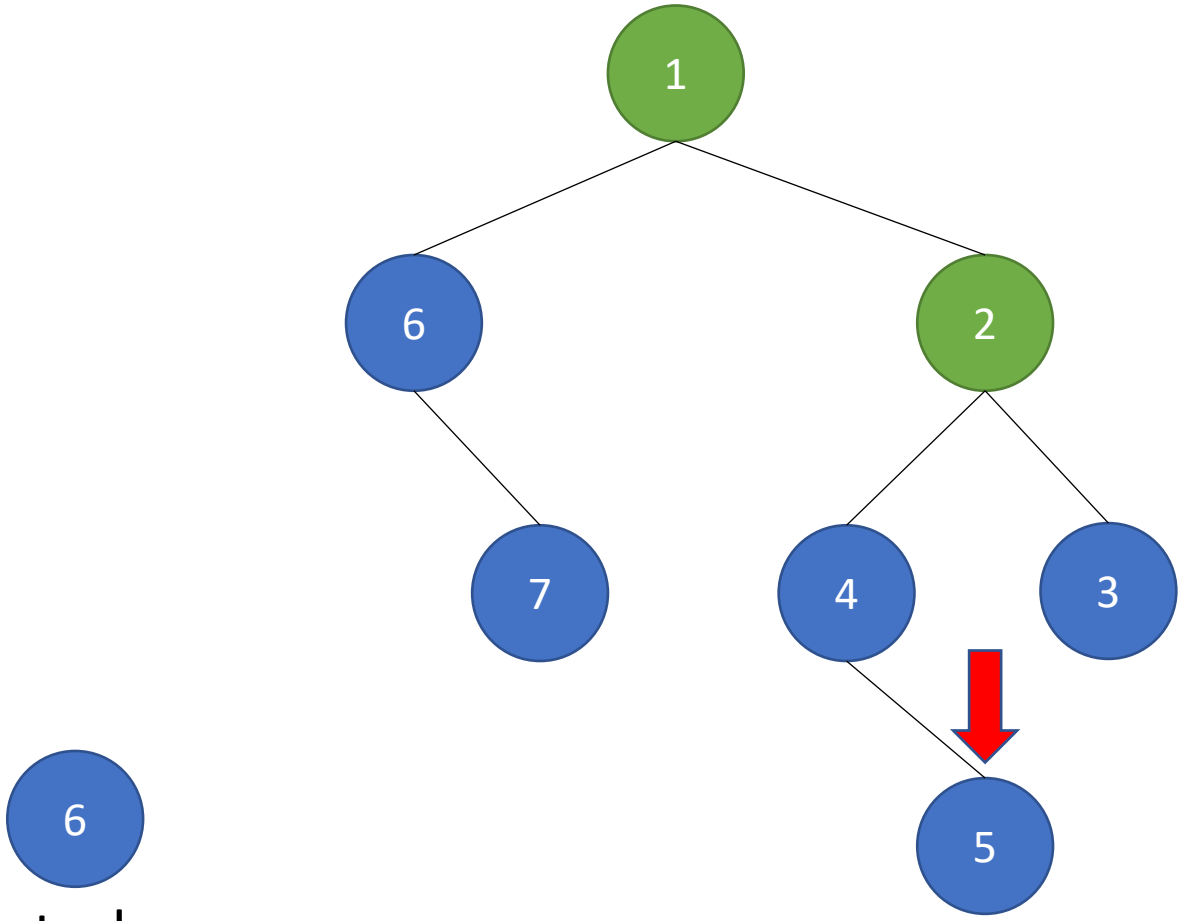
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

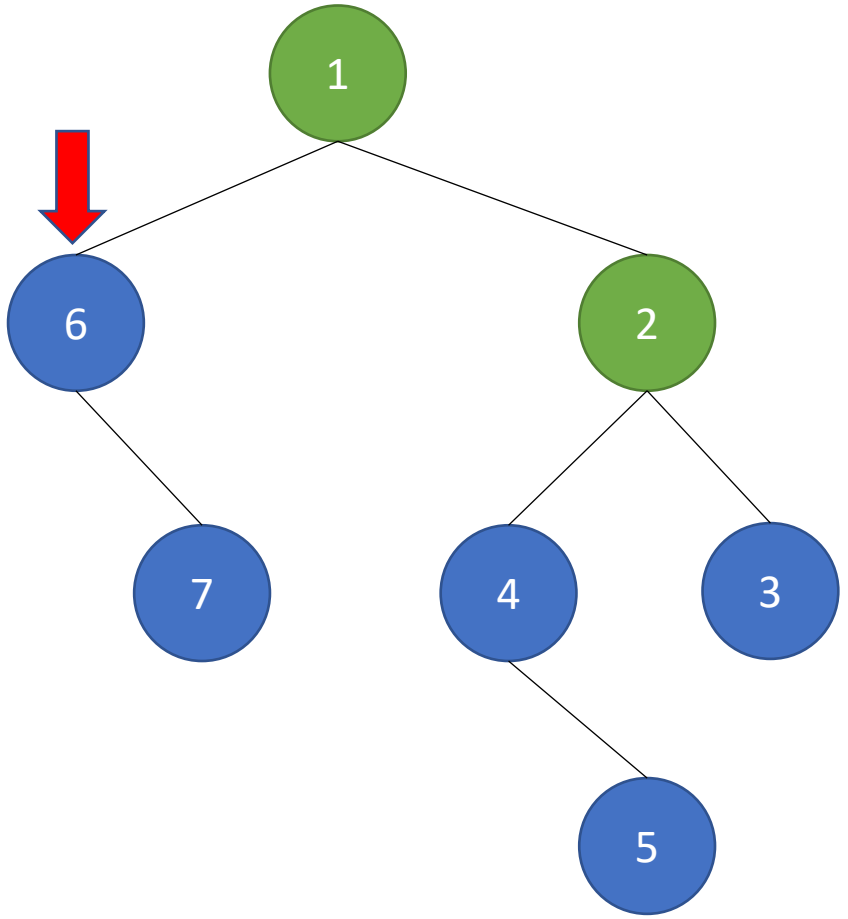
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer



stack

```
def has_grandchild(self, root):
    if root is None:
        return []

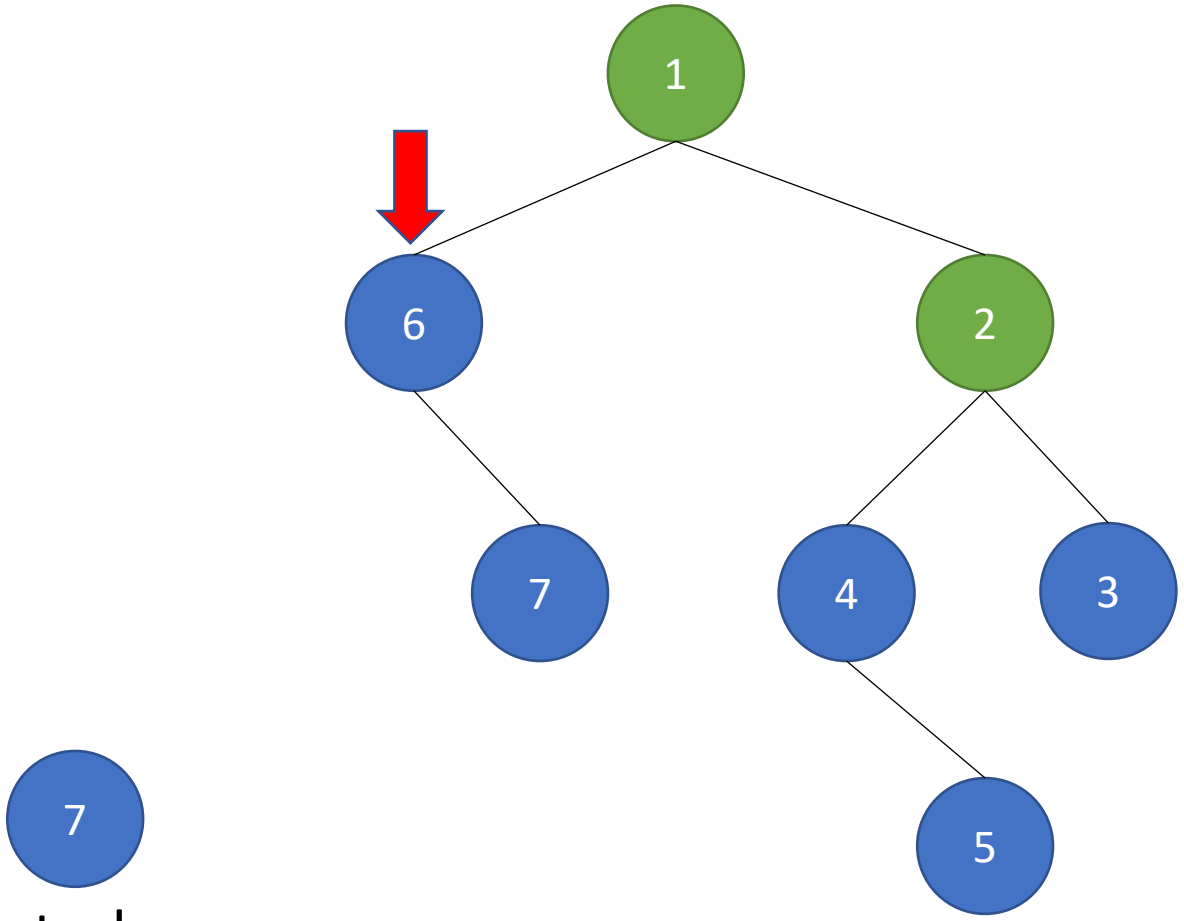
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer




```
def has_grandchild(self, root):
    if root is None:
        return []

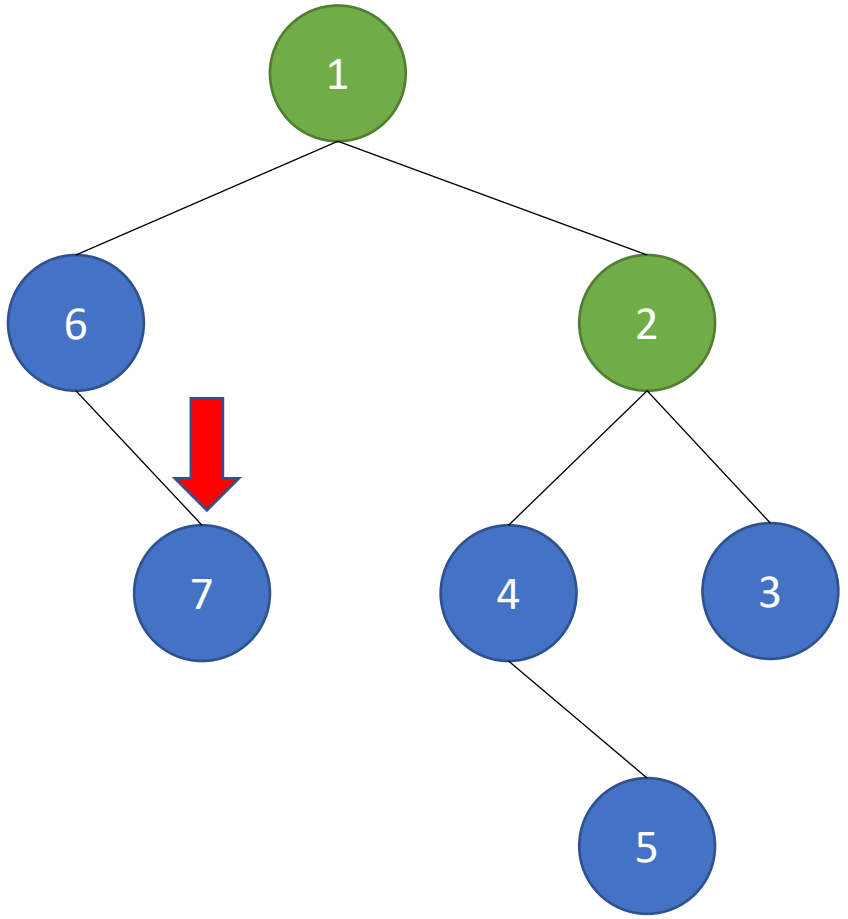
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer



stack

```
def has_grandchild(self, root):
    if root is None:
        return []

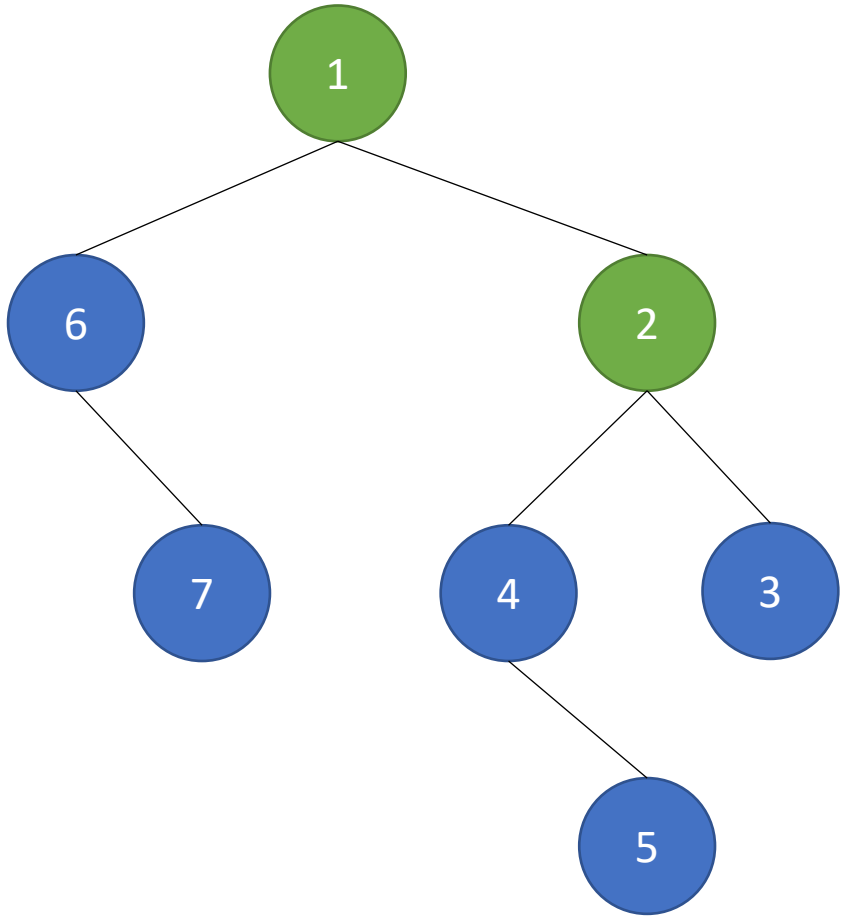
    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q2 (Normal) – Answer



stack

```
def has_grandchild(self, root):
    if root is None:
        return []

    result = []
    stack = [root]

    while stack:
        curr = stack.pop()
        has = False
        if curr.left:
            if curr.left.left or curr.left.right:
                has = True
            stack.append(curr.left)
        if curr.right:
            if curr.right.left or curr.right.right:
                has = True
            stack.append(curr.right)
        if has:
            result.append(curr.data)

    return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

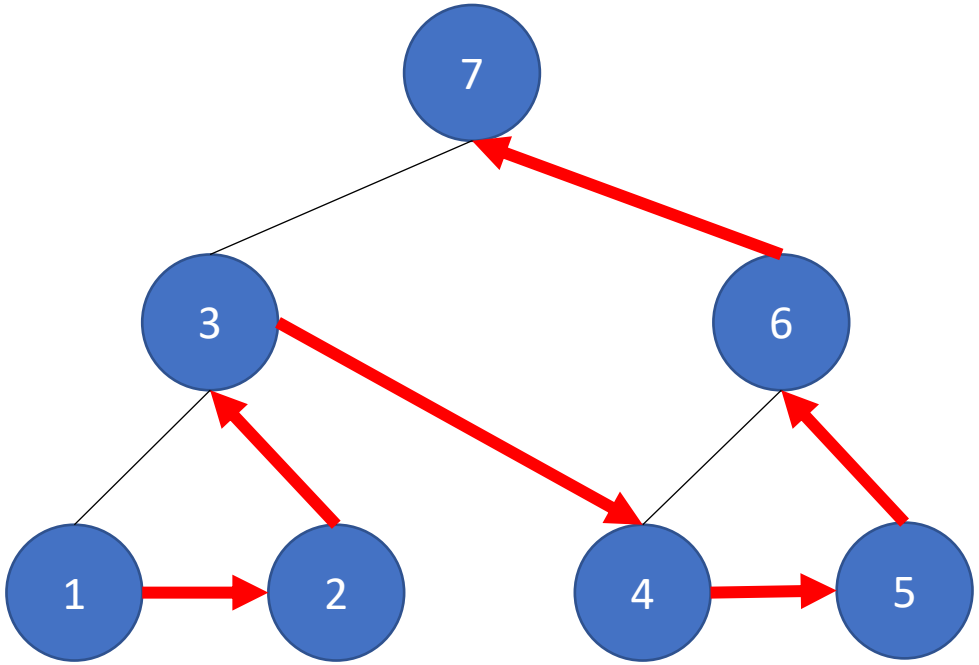
Write an iterative Python function postOrderIterativeS1() that prints the post-order traversal of a binary search tree using only one temporary stack inside the postOrderIterativeS1()function.

Note that you should only use push() or pop() operations when you add or remove integers from the stack.

Content Copyright Nanyang Technological University

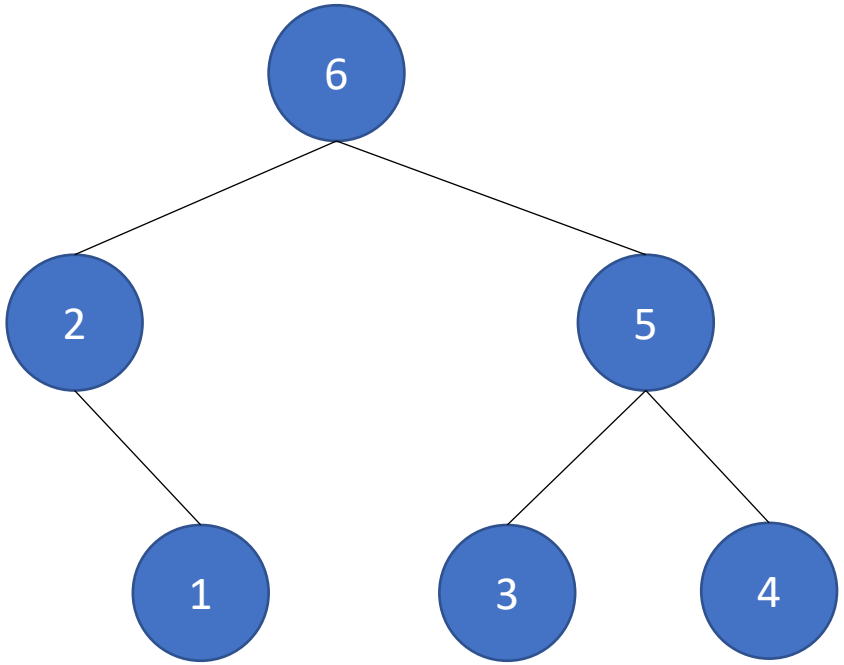
Tree Traversal

- Depth-First Search
  - Postorder traversal
  - Left, right, root



Content Copyright Nanyang Technological University

Q3 (Driver) – Answer




```
result = []
if root is None:
    return result
stack = Stack()
current = root
```

Content Copyright Nanyang Technological University

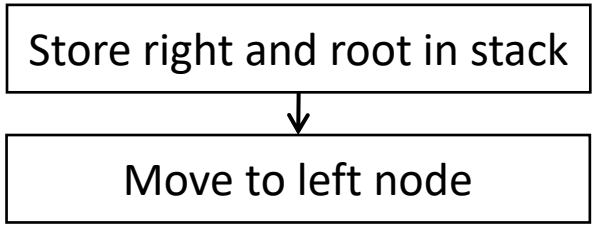
Q3 (Driver) – Answer

Store right and root in stack

```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

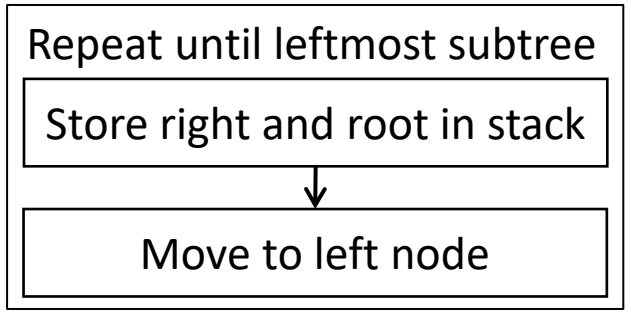
Q3 (Driver) – Answer




```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

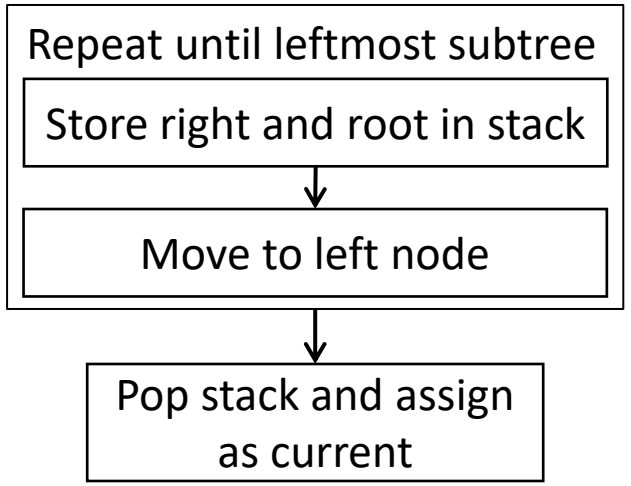
Q3 (Driver) – Answer




```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

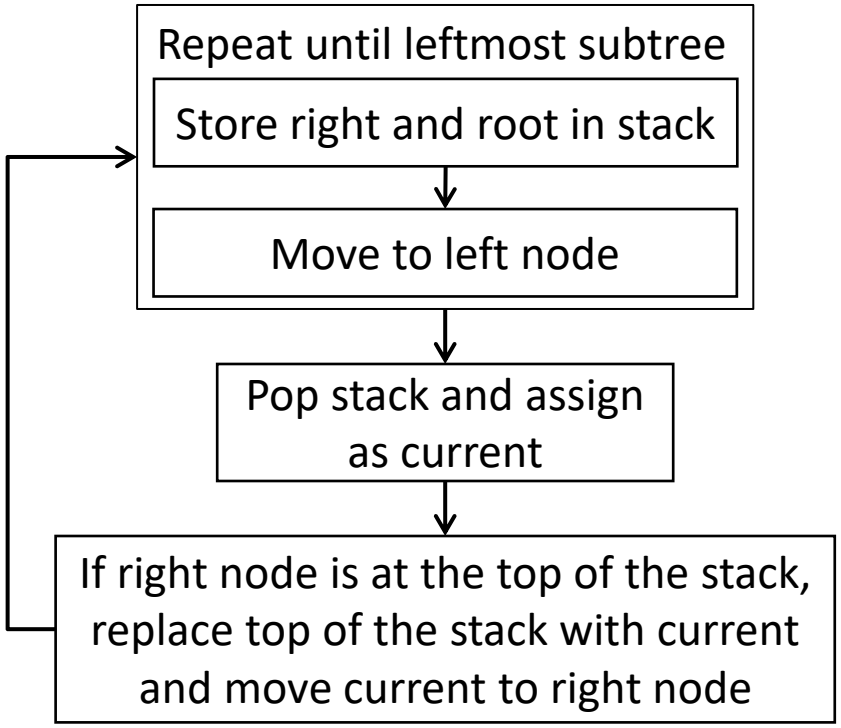
Q3 (Driver) – Answer




```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

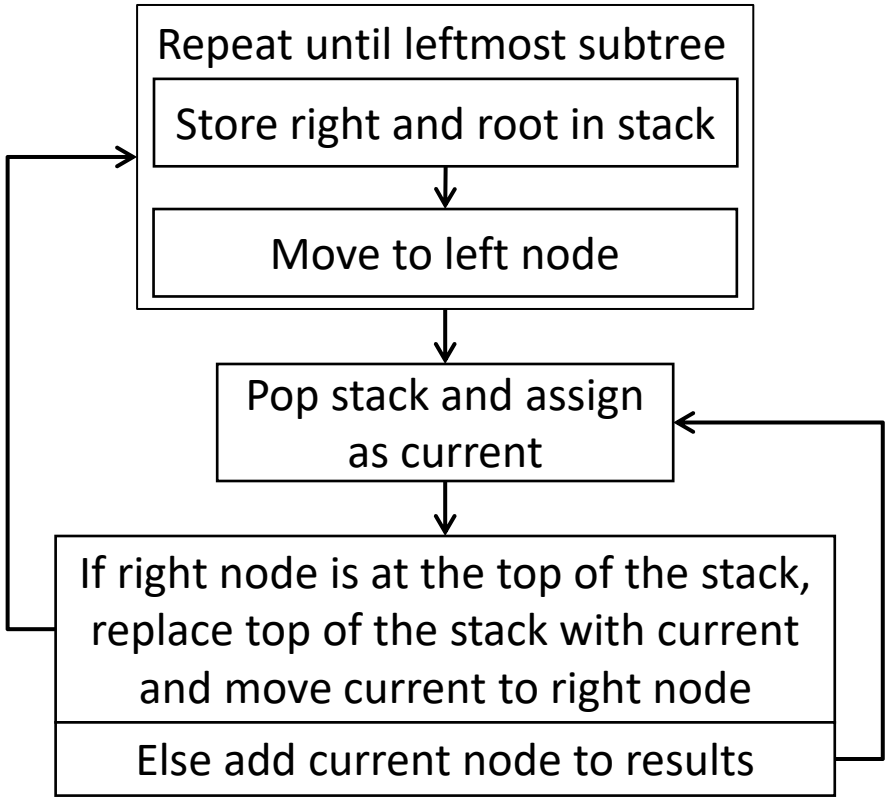
Q3 (Driver) – Answer




```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

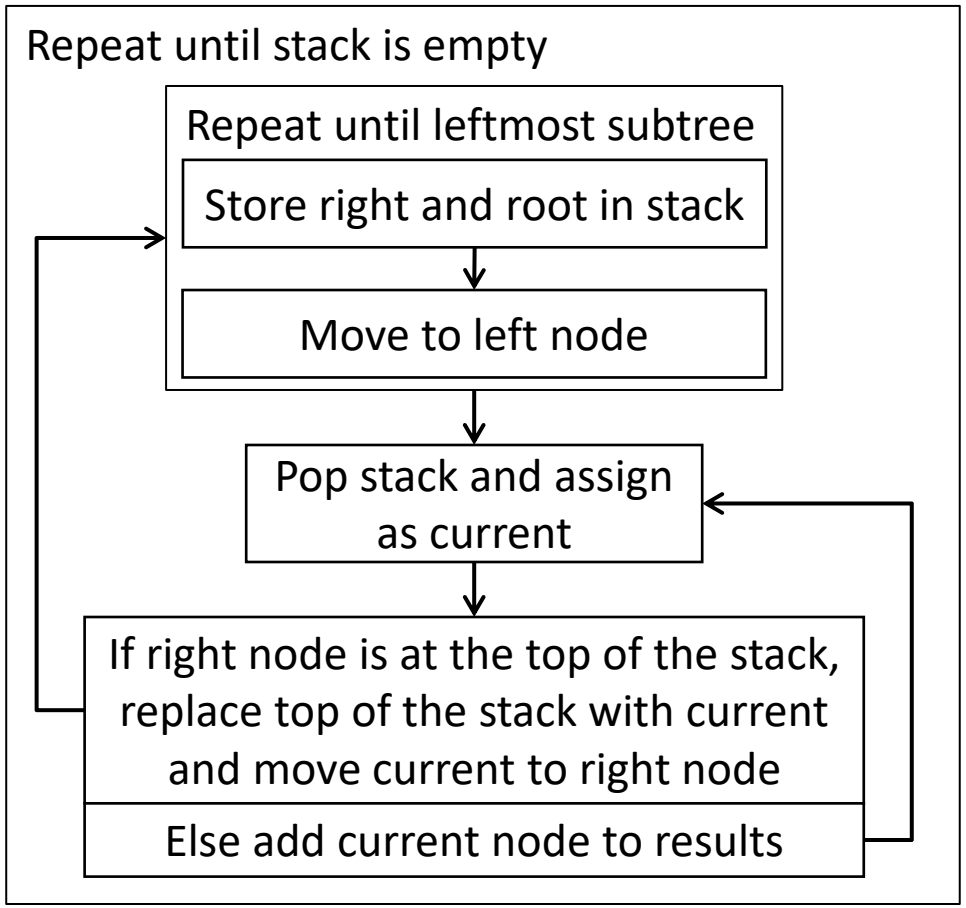
Q3 (Driver) – Answer




```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

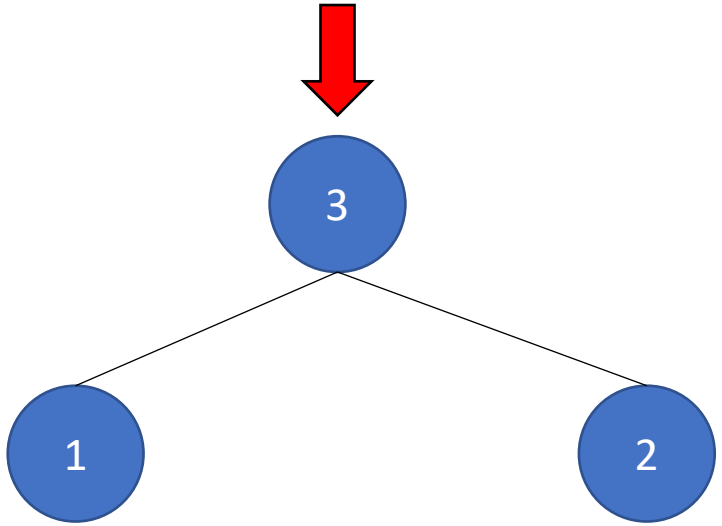
Q3 (Driver) – Answer

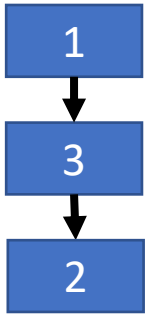



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

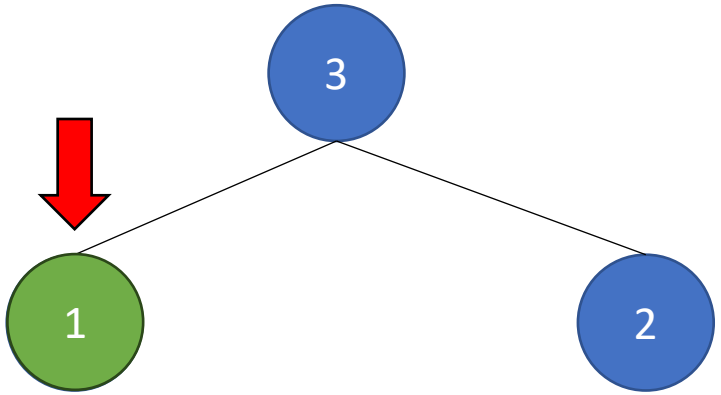


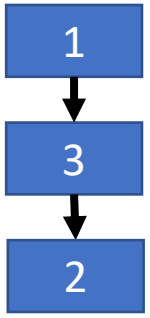



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

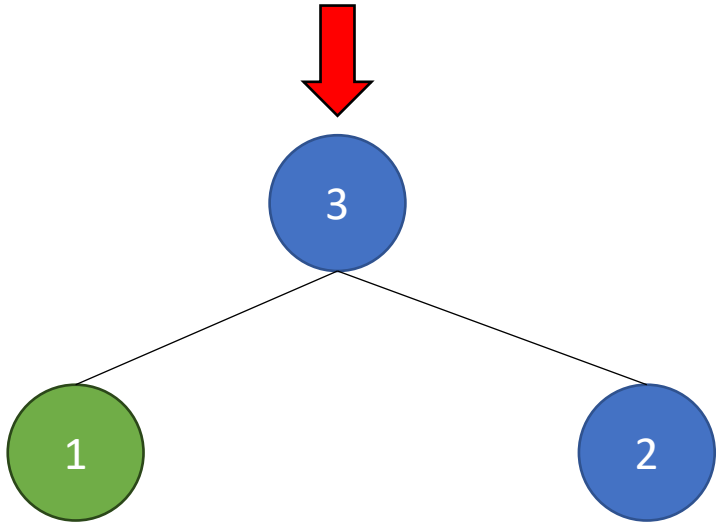


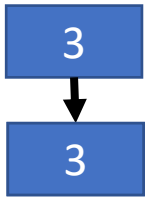



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

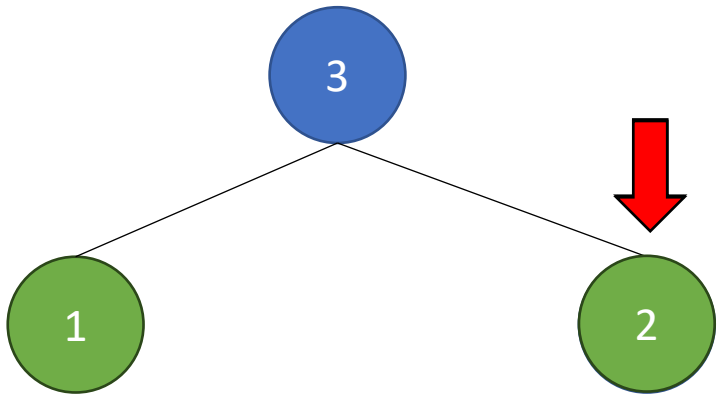


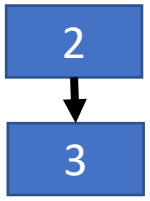



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

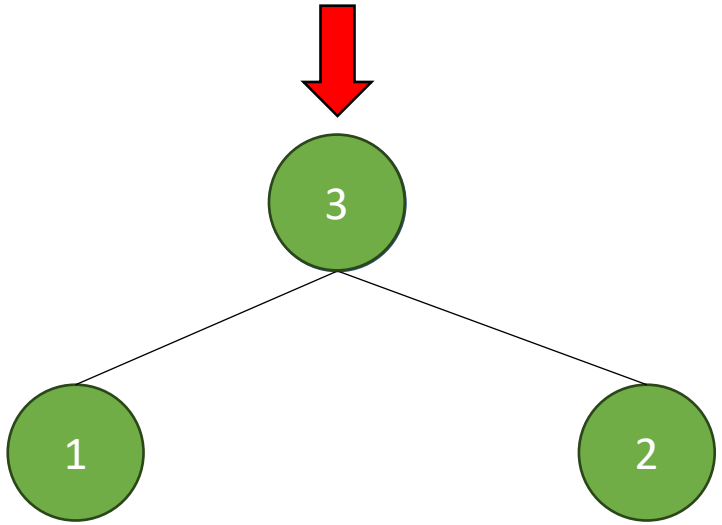





```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

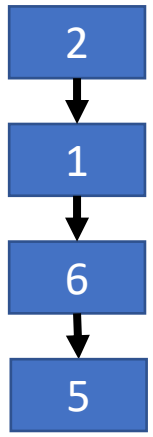


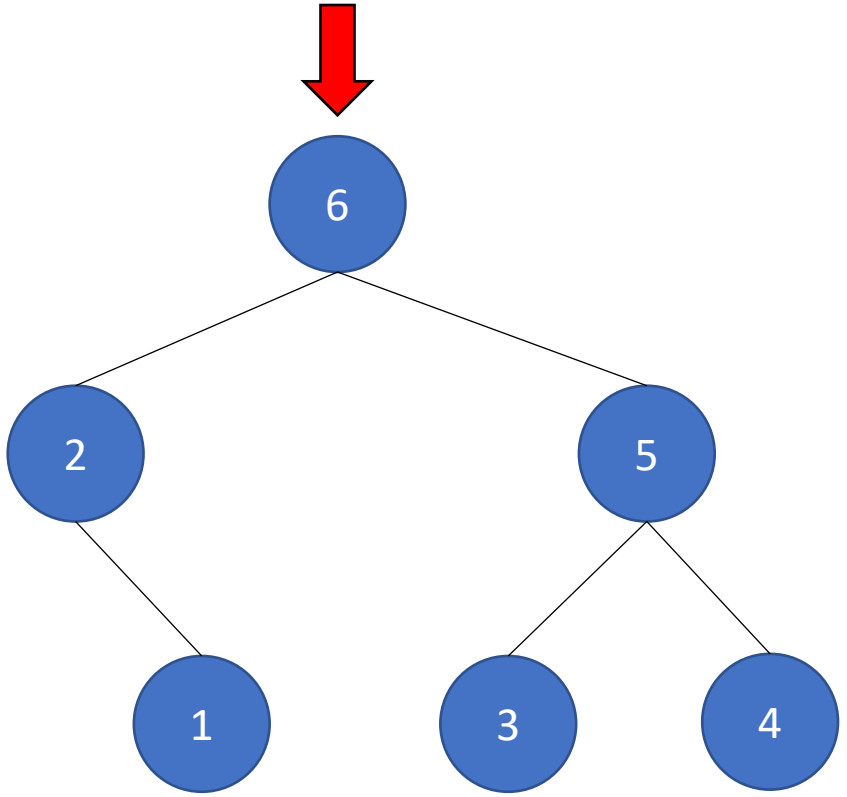

```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

3

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

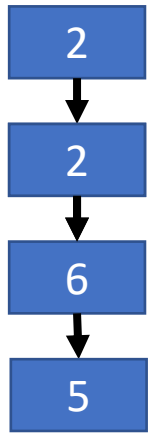


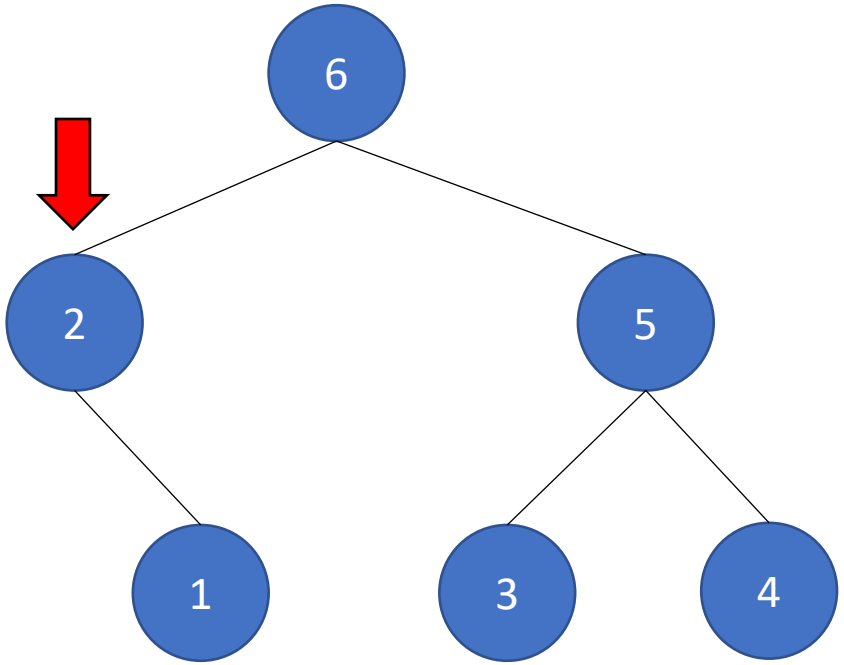



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

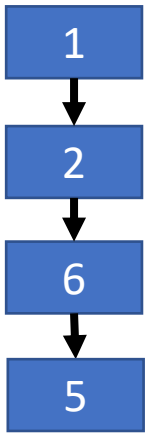


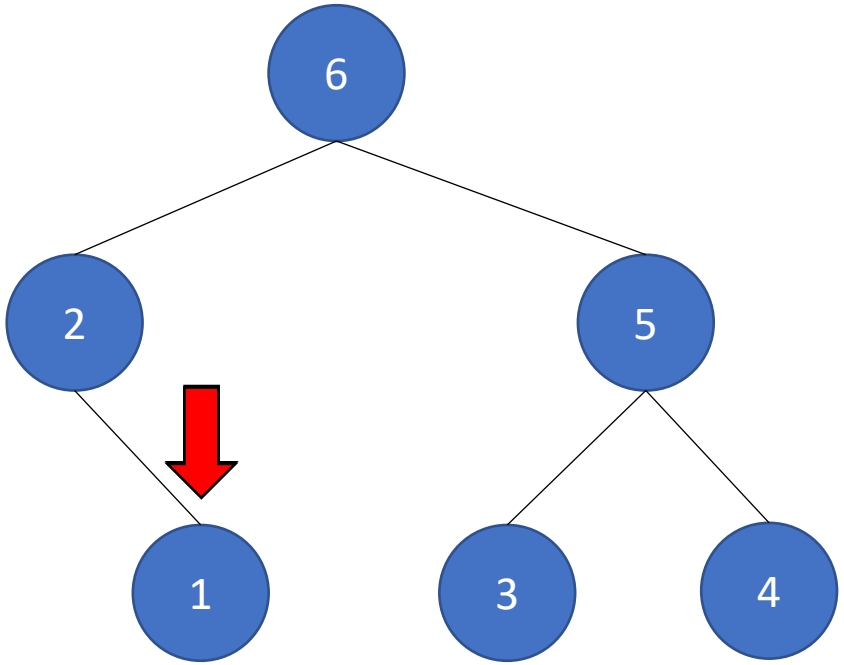



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

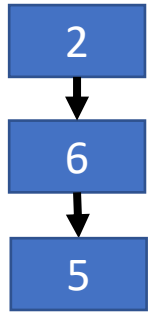


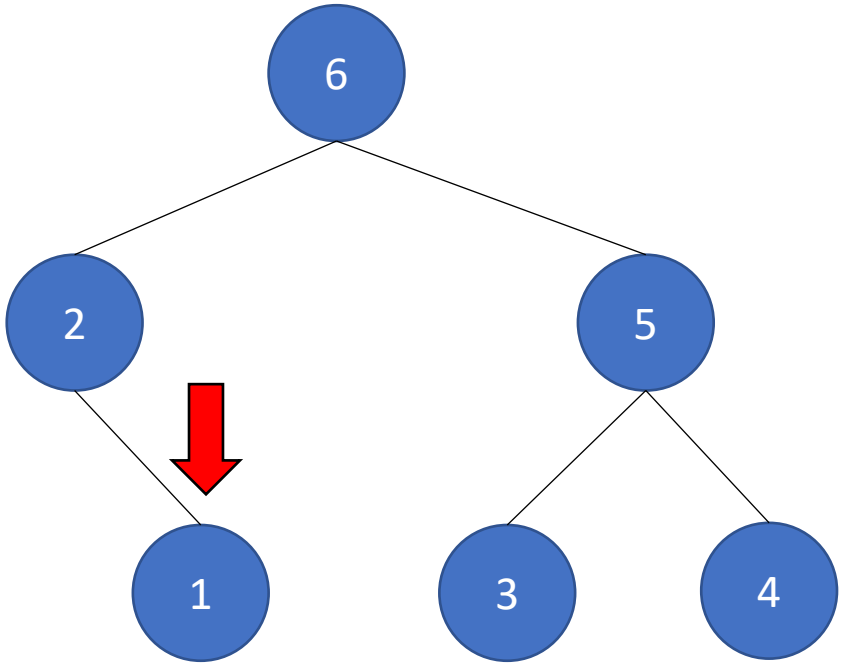



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

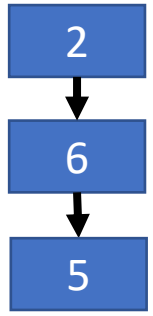


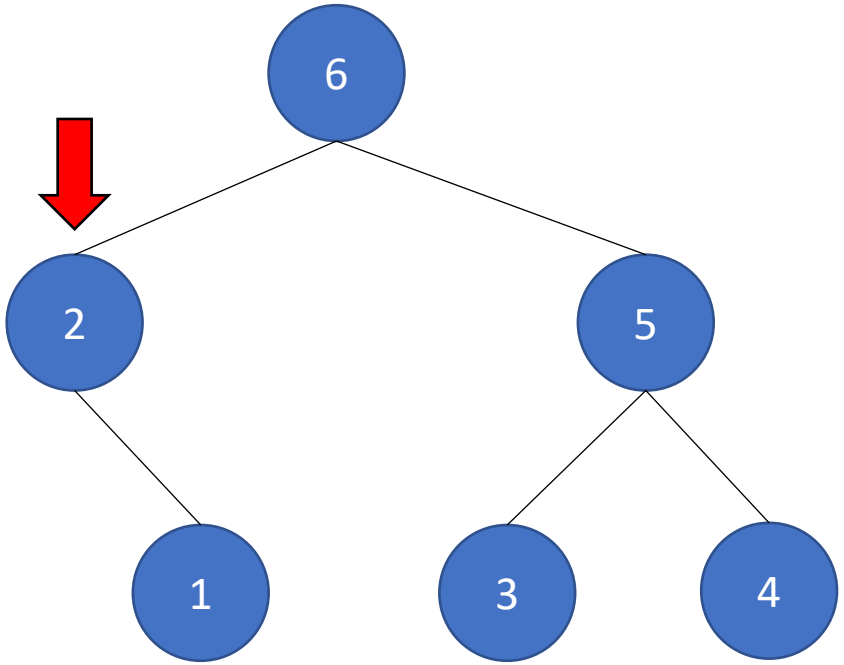



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

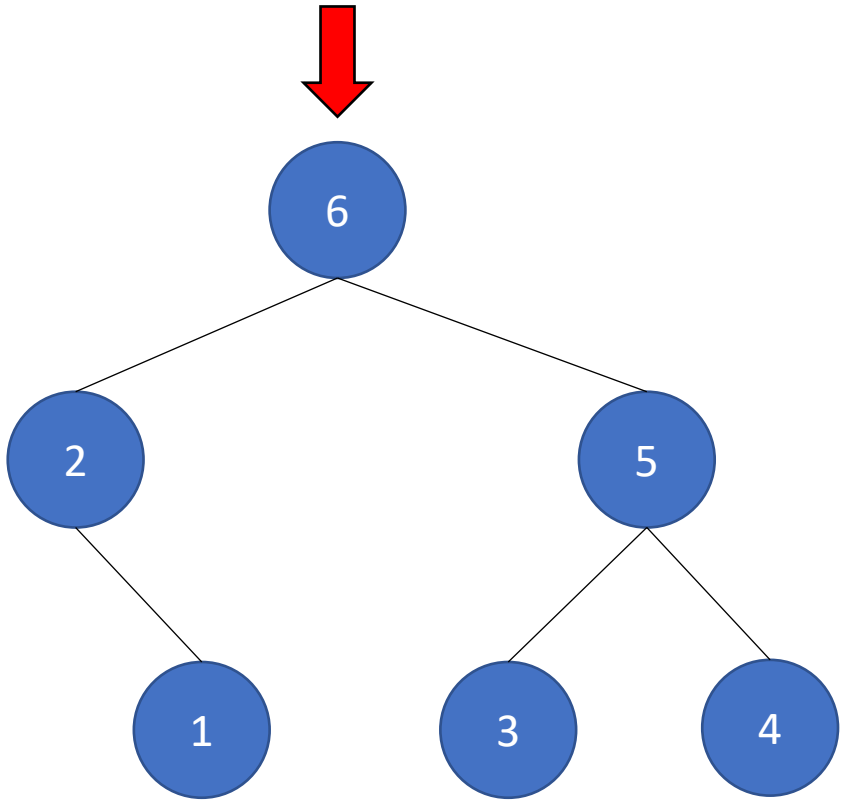


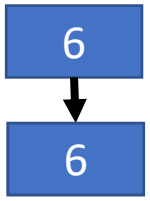



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

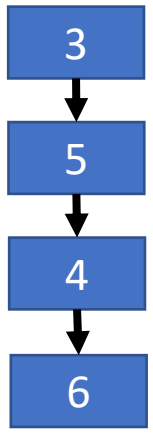


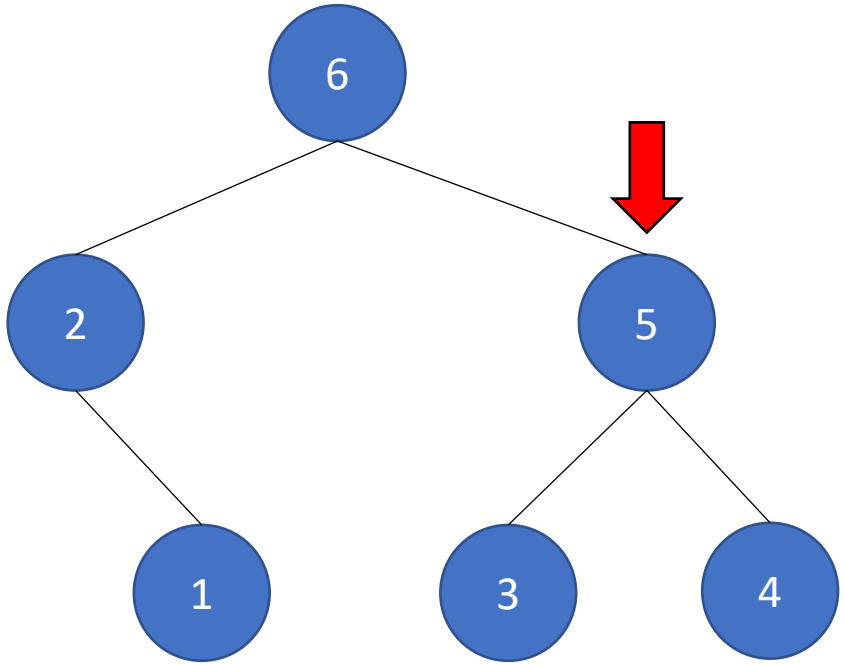



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

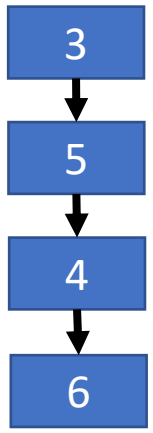


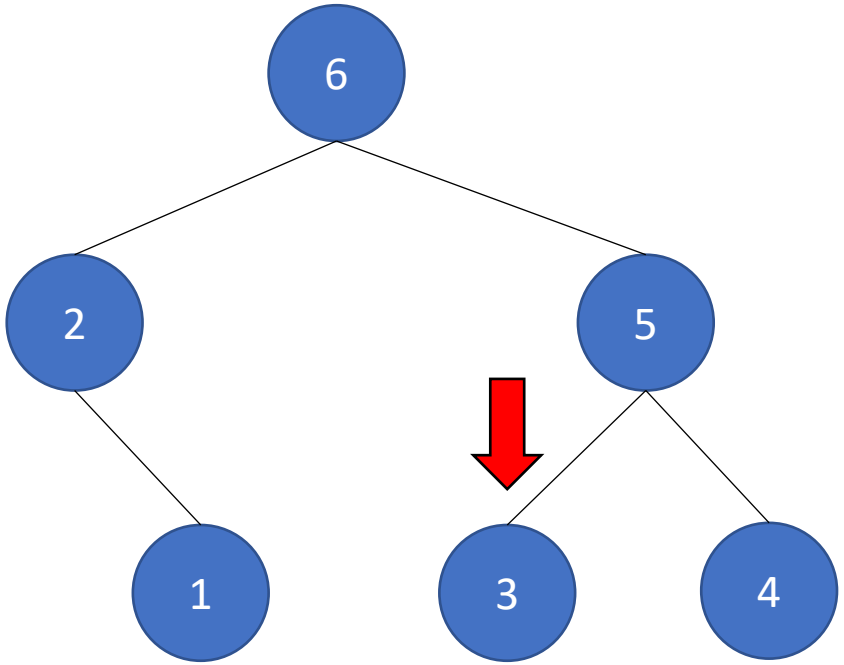



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer

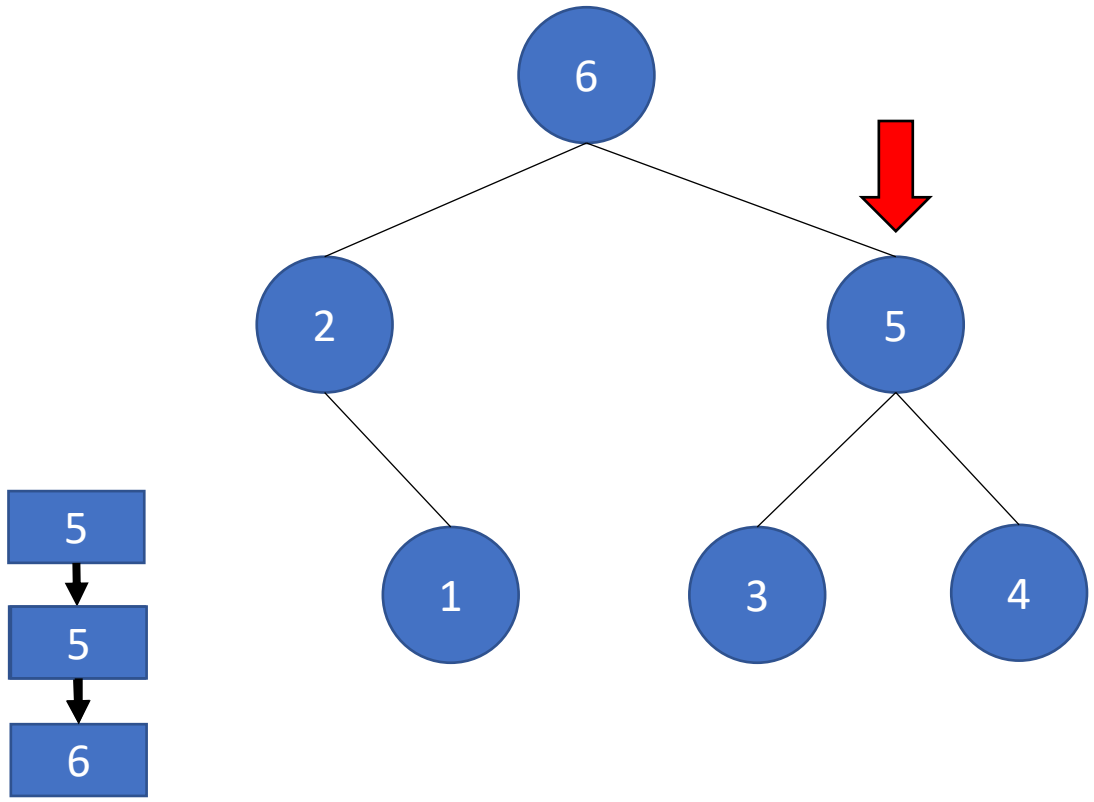





```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

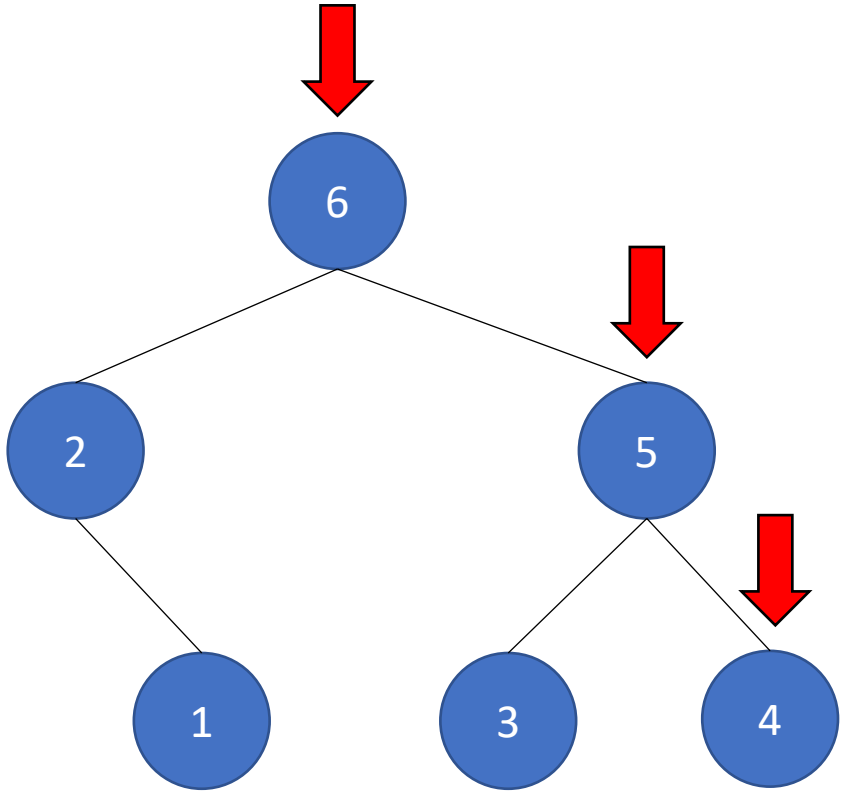
Q3 (Driver) – Answer

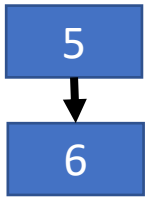



```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q3 (Driver) – Answer






```
while current or not is_empty(stack):
    while current:
        if current.right:
            push(stack, current.right)
        push(stack, current)
        current = current.left
    current = pop(stack)
    if current.right and not is_empty(stack)
    and peek(stack) == current.right:
        pop(stack)
        push(stack, current)
        current = current.right
    else:
        result.append(current.data)
        current = None
return result
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

Write a recursive Python function identical() to determine whether two binary trees are structurally identical, assuming the two binary trees as tree1 and tree2. This function returns TRUE if two binary trees are structurally identical; otherwise, it returns FALSE.

Note that two binary trees are structurally identical if they are both empty or if they are both non-empty and the left and the right subtrees are similar (they are made of nodes with the same values and arranged in the same way).

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

Write a recursive function with the following base cases:

- return True if both tree1 and tree2 are None
- return False if only one of the trees is None
- return False if the data of tree1 and tree2 is not the same

```
if tree1 is None and tree2 is None:
    return True
if tree1 is None or tree2 is None:
    return False
if tree1.data != tree2.data:
    return False
```

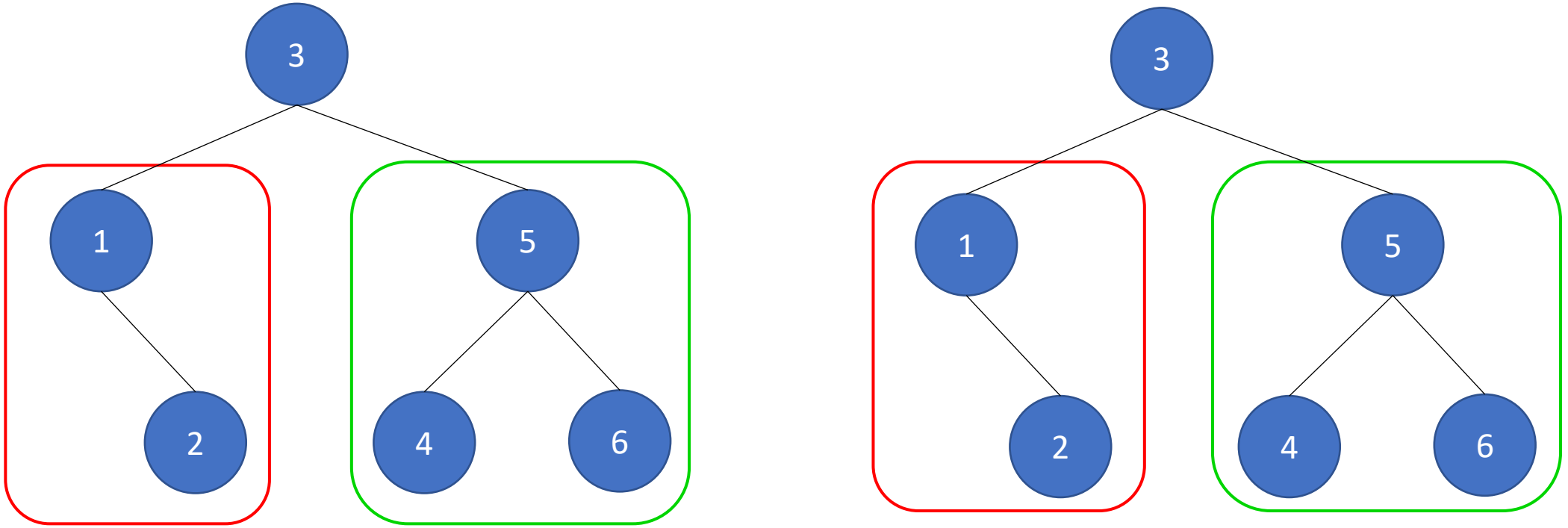
In return, recursively call the function using the left subtree of both trees and the right subtree of both trees

```
return self.identical(tree1.left, tree2.left) and
       self.identical(tree1.right, tree2.right)
```

Content Copyright Nanyang Technological University

Q4 (Navigator) – Answer

```
if tree1 is None and tree2 is None:
    return True
if tree1 is None or tree2 is None:
    return False
return (tree1.data == tree2.data and
        self.identical(tree1.left, tree2.left) and
        self.identical(tree1.right, tree2.right))
```



Content Copyright Nanyang Technological University